

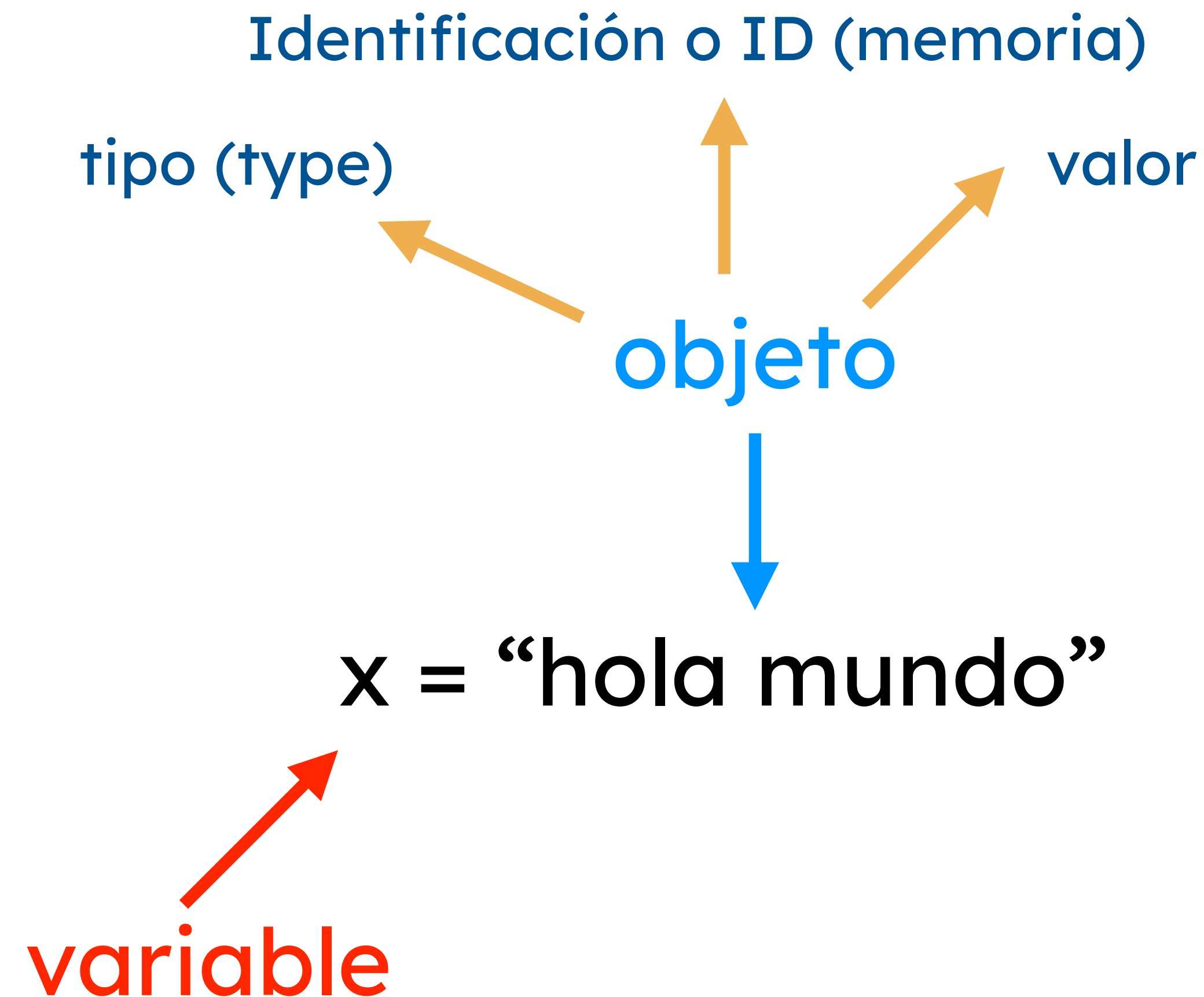
Python para ciencias de la atmósfera

Variables y objetos

```
x = "hola mundo"
```

TODO en python es
un objeto

Variables y objetos



Mutable vs immutable

- Los objetos se dividen en mutables e inmutables

mutables	inmutables
Se pueden modificar	No se pueden modificar

Tipos de objetos

Inmutable

Cadena o string

- Se escribe entre comillas simples o doble
- También se puede usar triple comilla

"hola mundo"

Tipos de objetos

Inmutable

Cadena o string

- Se escribe entre comillas simples o doble
- También se puede usar triple comilla

"hola mundo"

'hola mundo'

"""hola mundo"""

'''hola
mundo'''

Tipos de objetos

Inmutable

tuplas

- Se pueden escribir entre paréntesis
- Los paréntesis no son obligatorios
- Son colecciones de objetos
- Pueden contener cualquier tipo de objeto
- Los elementos (objetos) se separan por comas
- La tupla de 1 elemento lleva coma después del elemento

(1, 2, [1, 2, 3])

Tipos de objetos

Inmutable

tuplas

- Se pueden escribir entre paréntesis
- Los paréntesis no son obligatorios
- Son colecciones de objetos
- Pueden contener cualquier tipo de objeto
- Los elementos (objetos) se separan por comas
- La tupla de 1 elemento lleva coma después del elemento

(1, 2, [1, 2, 3])

1, 2, [1, 2, 3]

Tipos de objetos

Inmutable

tuplas

- Se pueden escribir entre paréntesis
- Los paréntesis no son obligatorios
- Son colecciones de objetos
- Pueden contener cualquier tipo de objeto
- Los elementos (objetos) se separan por comas
- La tupla de 1 elemento lleva coma después del elemento

(1, 2, [1, 2, 3])

1, 2, [1, 2, 3]

1,

Tipos de objetos

Mutable

Listas

- Se escriben entre corchetes cuadrados
- Son colecciones de objetos
- Pueden contener cualquier tipo de objeto
- Los elementos (objetos) se separan por comas

```
[1, 2, [1, 2, 3]]  
[1]
```

Tipos de objetos

Mutable

Diccionarios

- Se escriben entre llaves
- Son colecciones de elementos
- Los elementos del diccionario son pares clave: valor
- Las claves y los valores son objetos
- Los elementos se separan por comas
- Las claves deben ser objetos inmutables
- Los valores pueden ser cualquier tipo de objeto

```
{1: 'hola', 'dos': 2, (1,2): [1,2,3]}
```

Tipos de objetos

Mutable

Diccionarios

- Se escriben entre llaves
- Son colecciones de elementos
- Los elementos del diccionario son pares clave: valor
- Las claves y los valores son objetos
- Los elementos se separan por comas
- Las claves deben ser objetos mutables
- Los valores pueden ser cualquier tipo de objeto

`{1: 'hola', 'dos': 2, (1,2): [1,2,3]}`

↓ ↓ ↓

clave clave clave

Tipos de objetos

Mutable

Diccionarios

- Se escriben entre llaves
- Son colecciones de elementos
- Los elementos del diccionario son pares clave: valor
- Las claves y los valores son objetos
- Los elementos se separan por comas
- Las claves deben ser objetos mutables
- Los valores pueden ser cualquier tipo de objeto

{1: 'hola', 'dos': 2, (1, 2): [1, 2, 3]}

The diagram illustrates the structure of a Python dictionary. It shows a set of curly braces containing three key-value pairs separated by commas. Each part of the pairs is color-coded: keys are in green or blue, and values are in brown or green. Below each part, a colored arrow points to a label: blue arrows point to 'clave' (key) and red arrows point to 'valores' (value).

Element	Type	Label
1	clave	clave
'hola'	valores	valores
'dos'	clave	clave
2	valores	valores
(1, 2)	clave	clave
[1, 2, 3]	valores	valores

Tipos de objetos

Mutable

Diccionarios

- Se escriben entre llaves
- Son colecciones de elementos
- Los elementos del diccionario son pares clave: valor
- Las claves y los valores son objetos
- Los elementos se separan por comas
- Las claves deben ser objetos mutables
- Los valores pueden ser cualquier tipo de objeto

```
{1: 'hola', 'dos': 2, (1,2): [1,2,3]}
```

```
{}
```

Tipos de objetos

Mutable

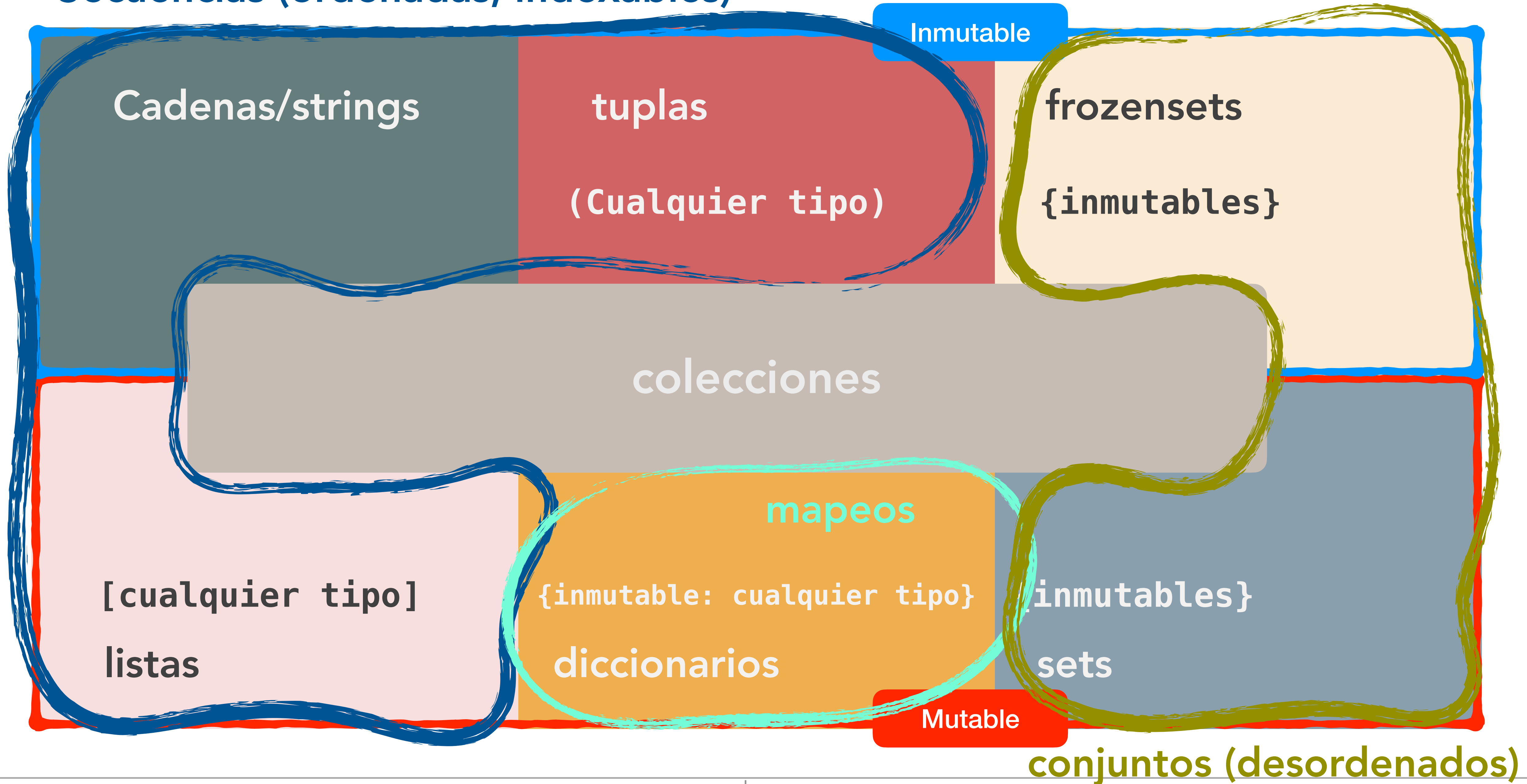
sets

- Se escriben entre llaves
- Son colecciones de elementos
- Los elementos deben ser inmutables
- No hay elementos repetidos
- Un set vacío se define con la función set()

{1, 2, (1, 2)}

Secuencias (ordenadas, indexables)

17



Tipos de objetos

Inmutables

números
(int, float, complex, bool)

cadena
(string)

tuplas

frozenset

Secuencias (ordenadas, indexables)

conjuntos (desordenados)

mutables

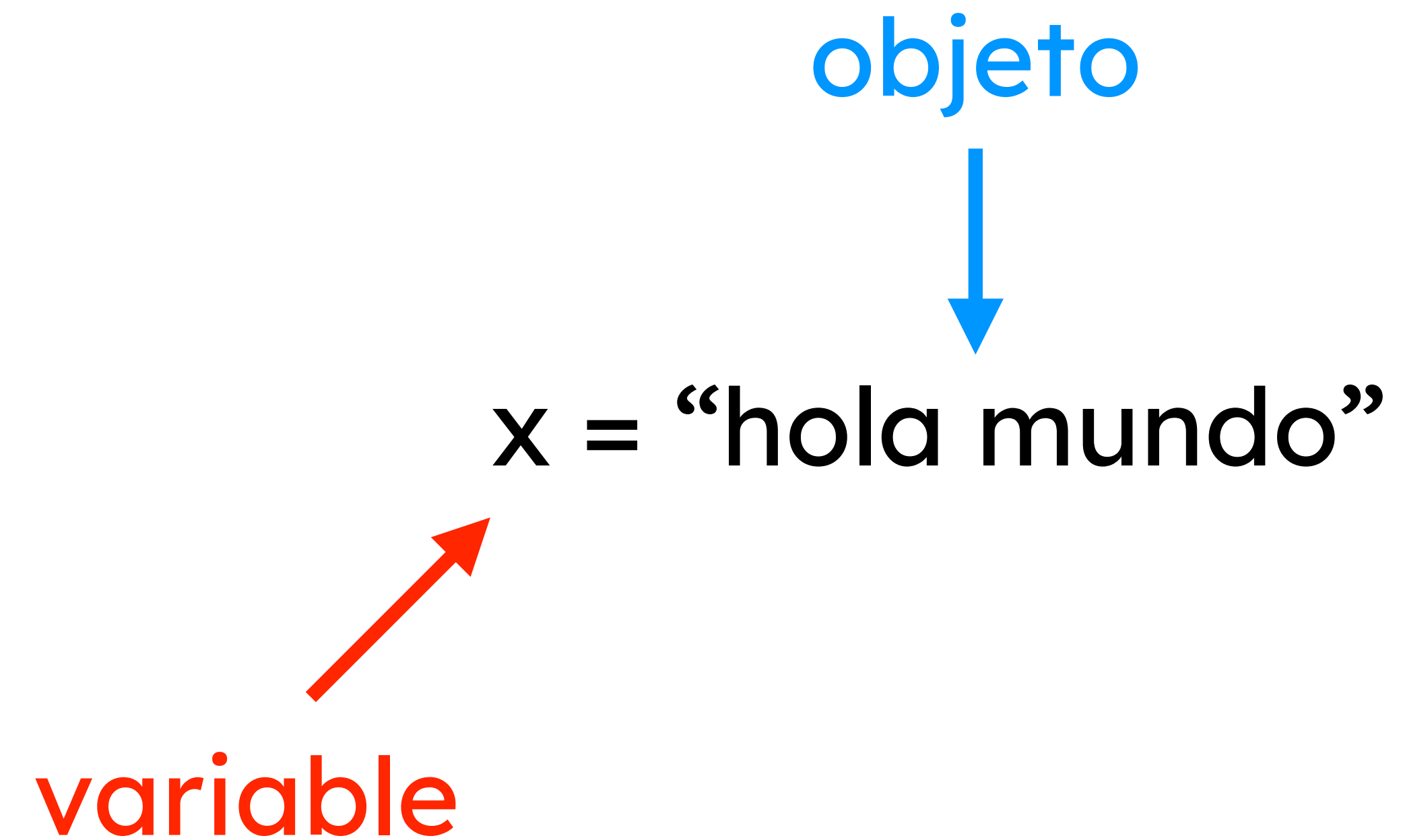
listas

diccionarios

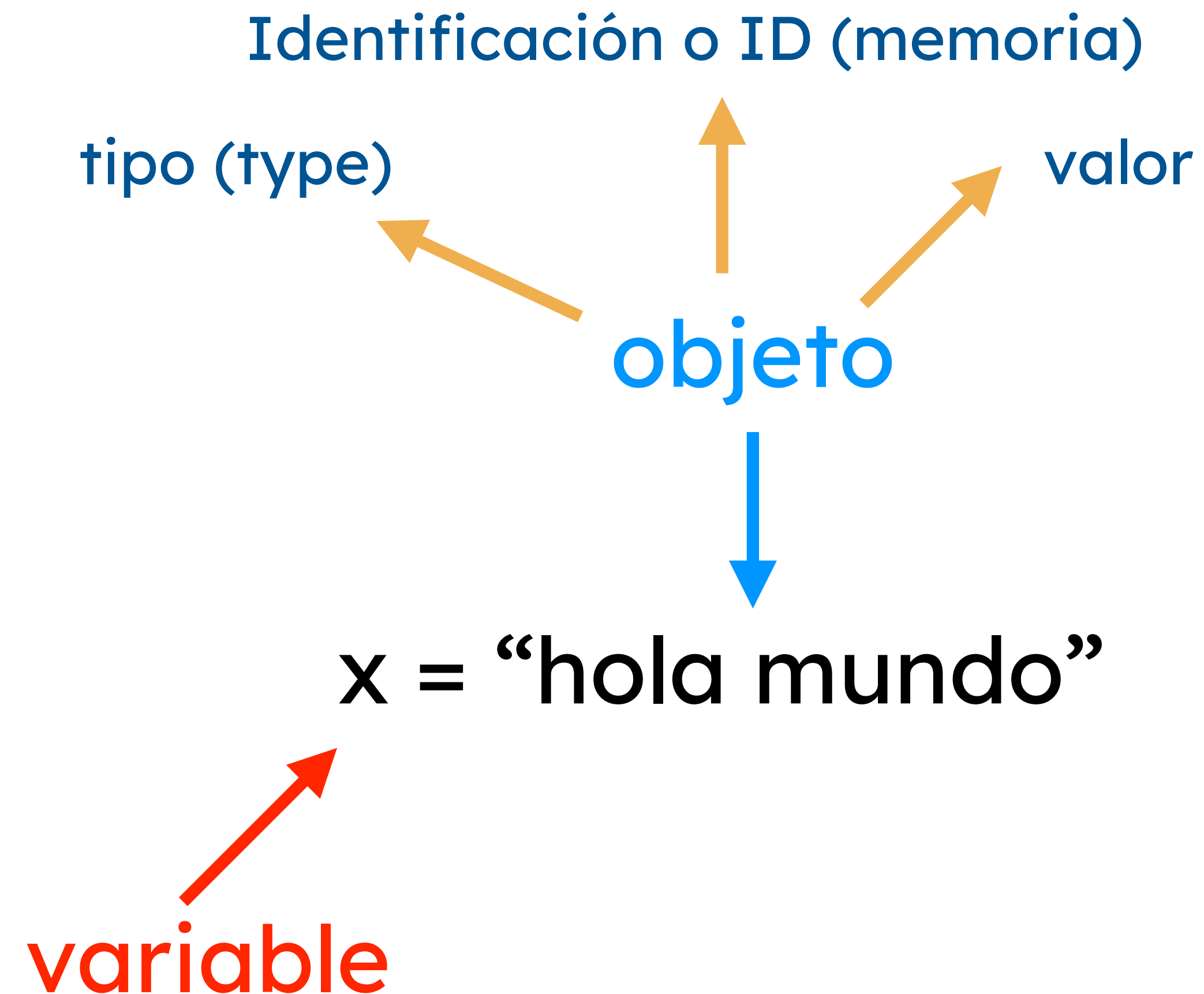
sets

Colecciones

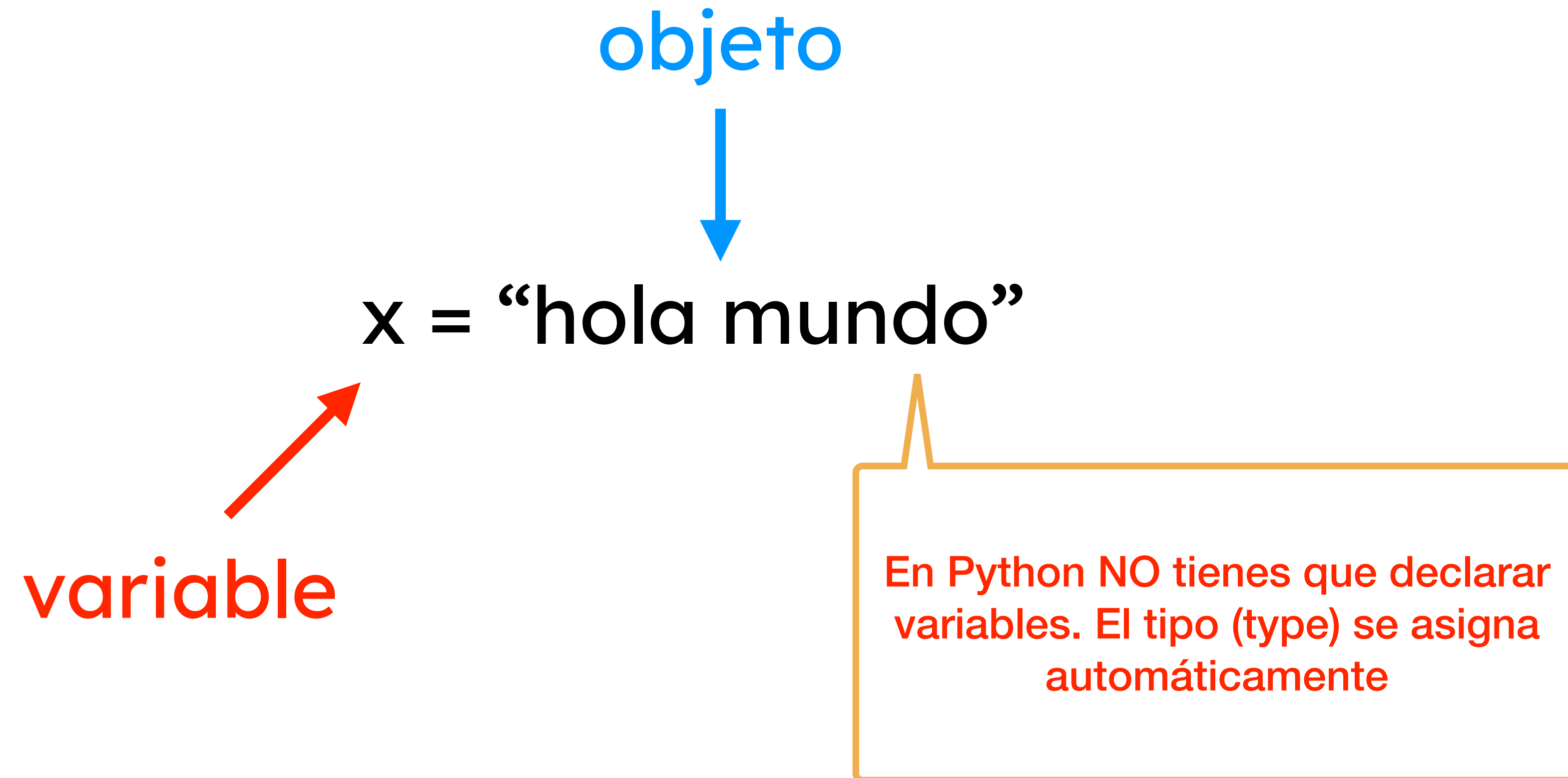
Variables y objetos



Variables y objetos

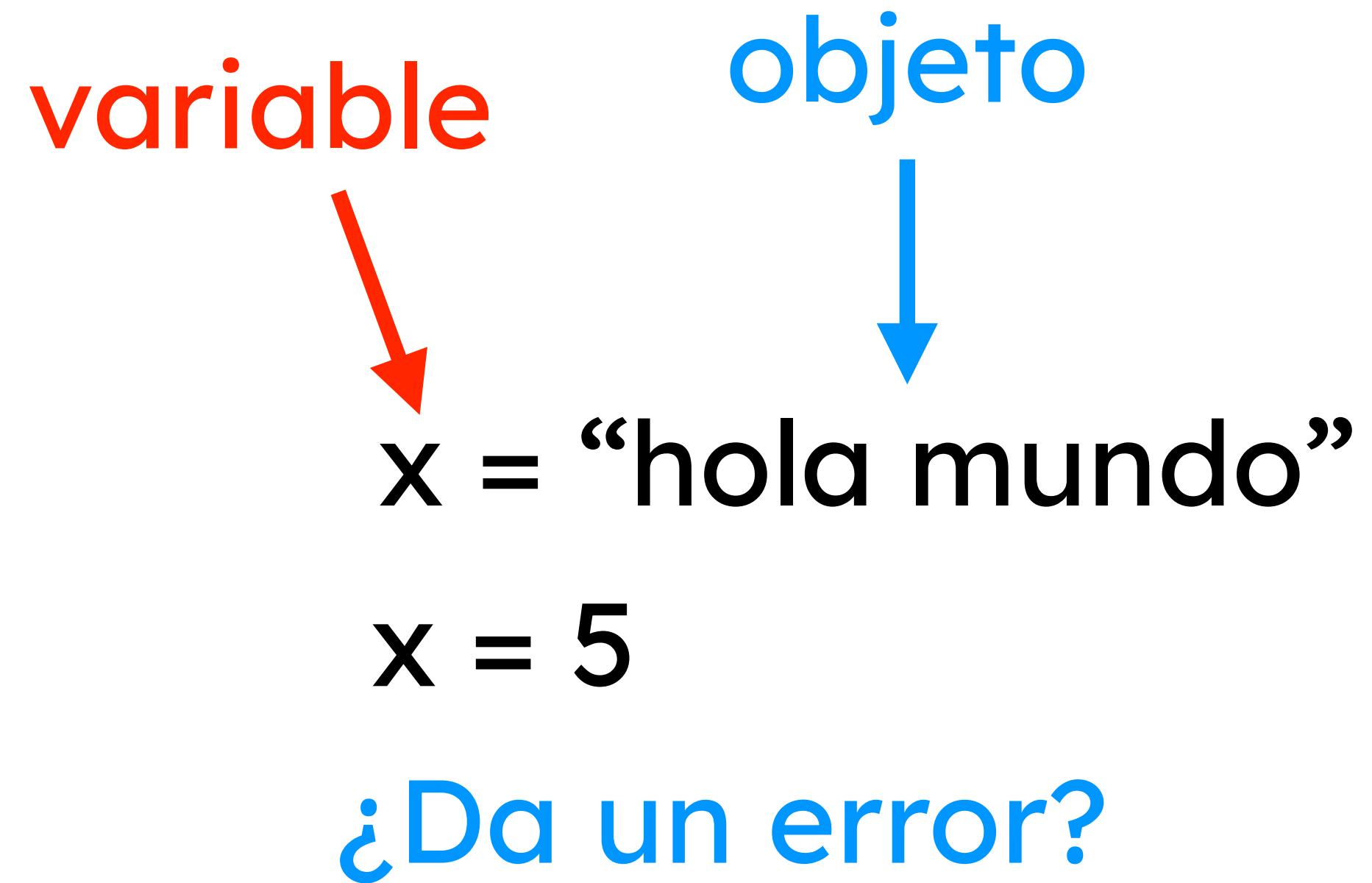


Variables y objetos



Variables y objetos

variable objeto



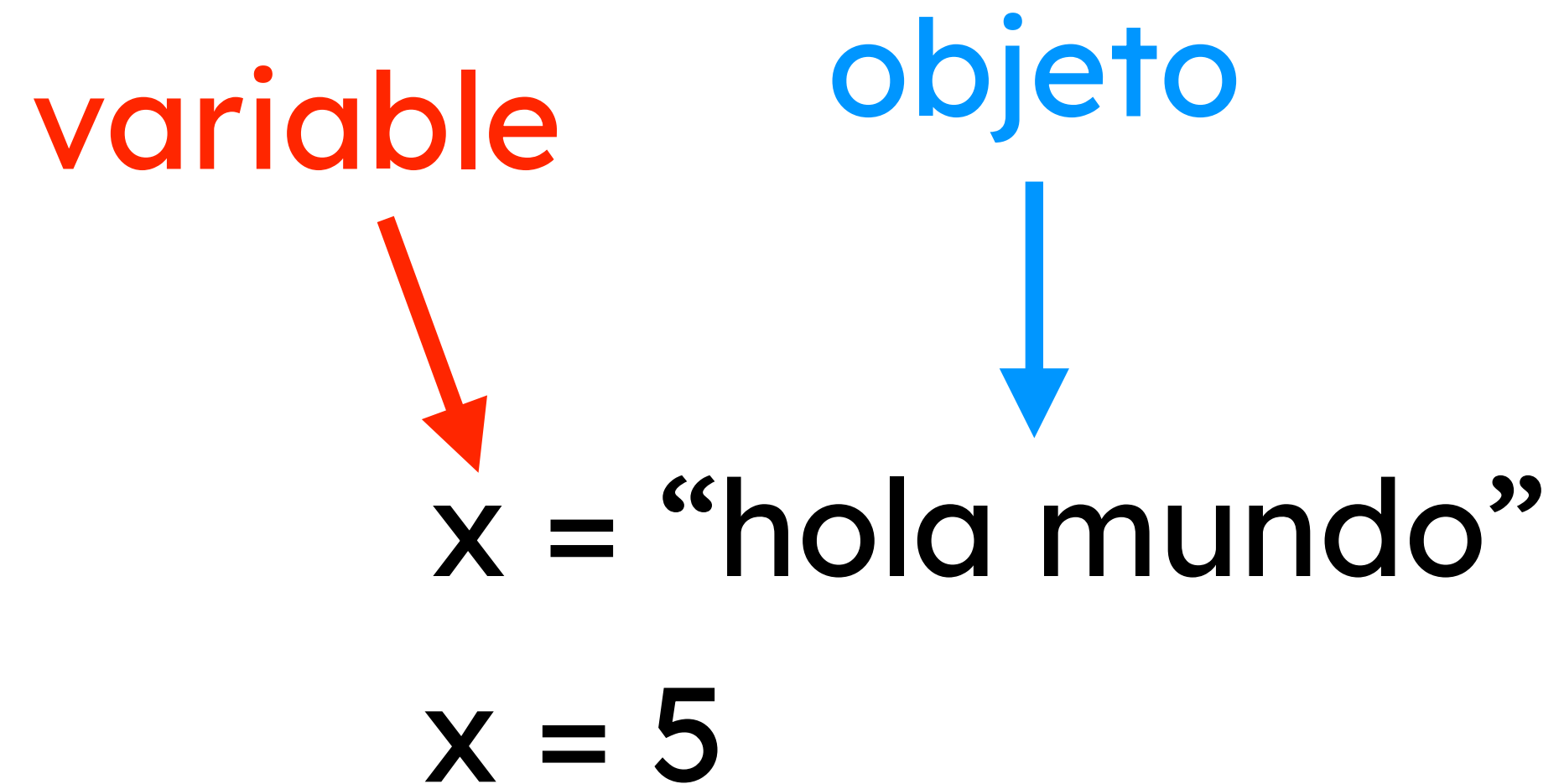
A red arrow points from the word 'variable' to the variable 'x' in the first line of code. A blue arrow points from the word 'objeto' to the string value '“hola mundo”' in the first line of code.

```
x = “hola mundo”  
x = 5
```

¿Da un error?

Variables y objetos

variable objeto



x = "hola mundo"

x = 5

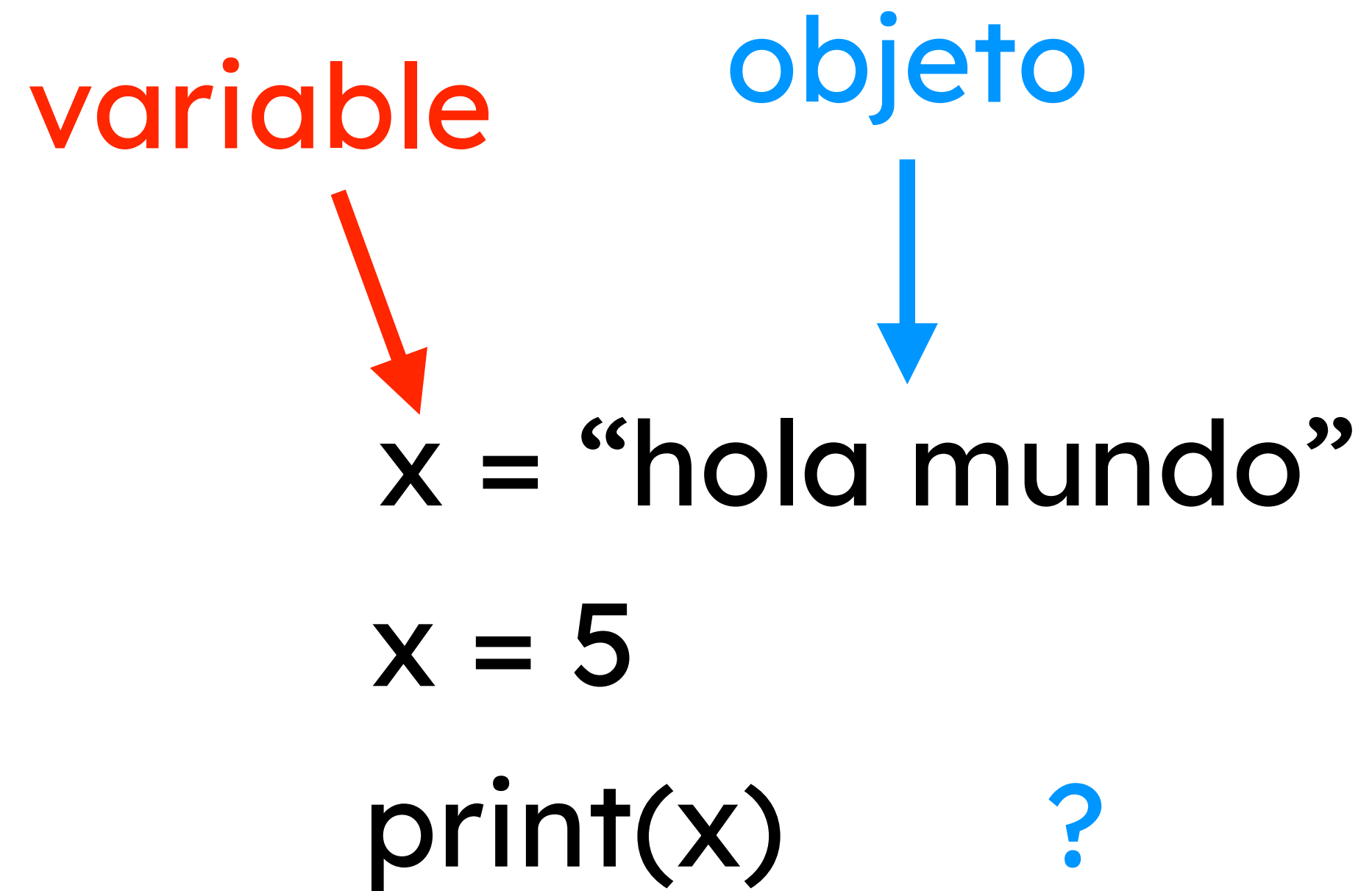
The diagram illustrates the relationship between variables and objects in Python. A red arrow points from the word 'variable' to the variable 'x' in the first line of code, and a blue arrow points from the word 'objeto' to the string object 'hola mundo' in the same line. The second line of code shows the variable 'x' being reassigned to a new object, the integer '5'.

¿Da un error?

En otros lenguajes de programación no es correcto, en python no hay problema

Variables y objetos

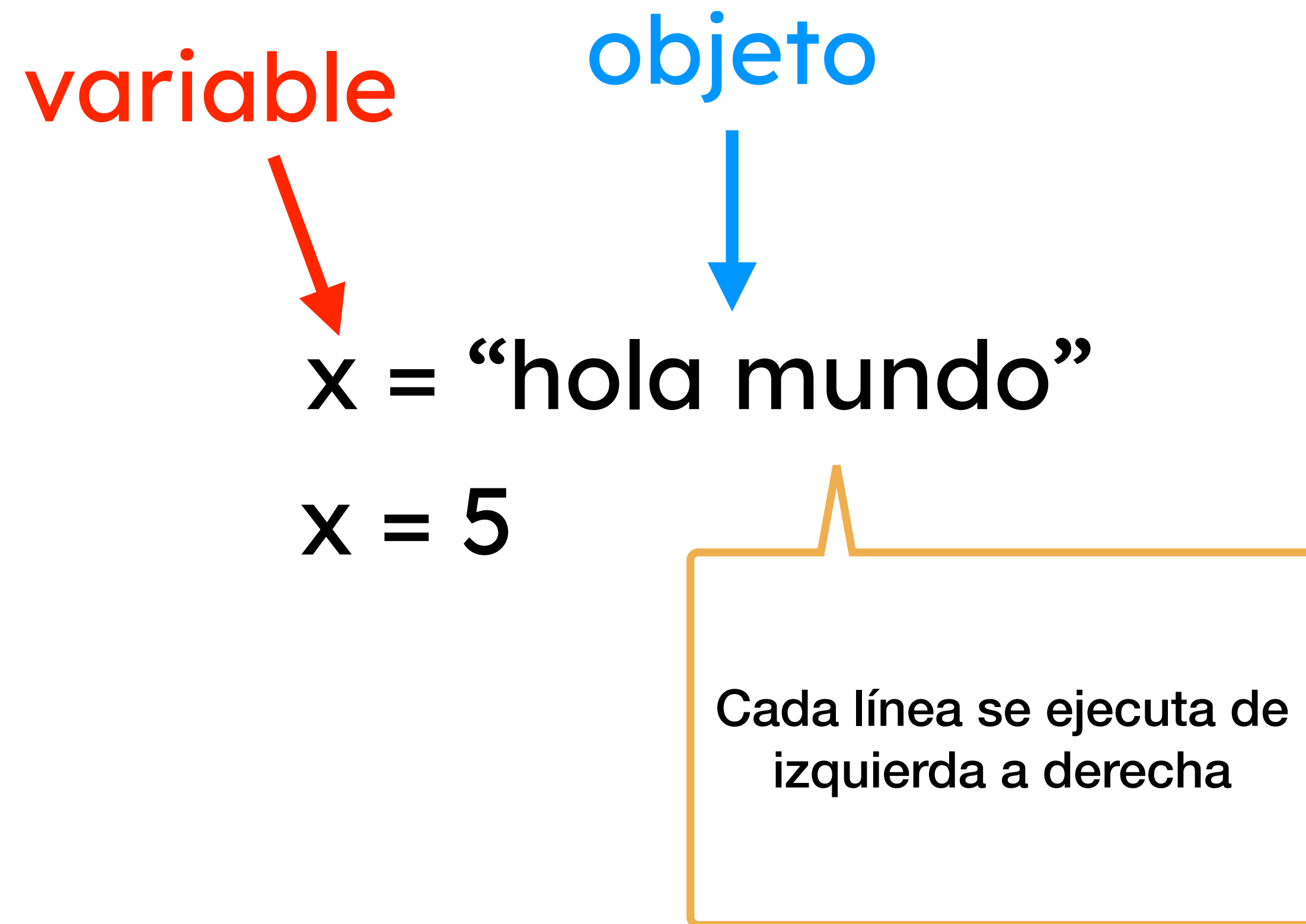
variable objeto



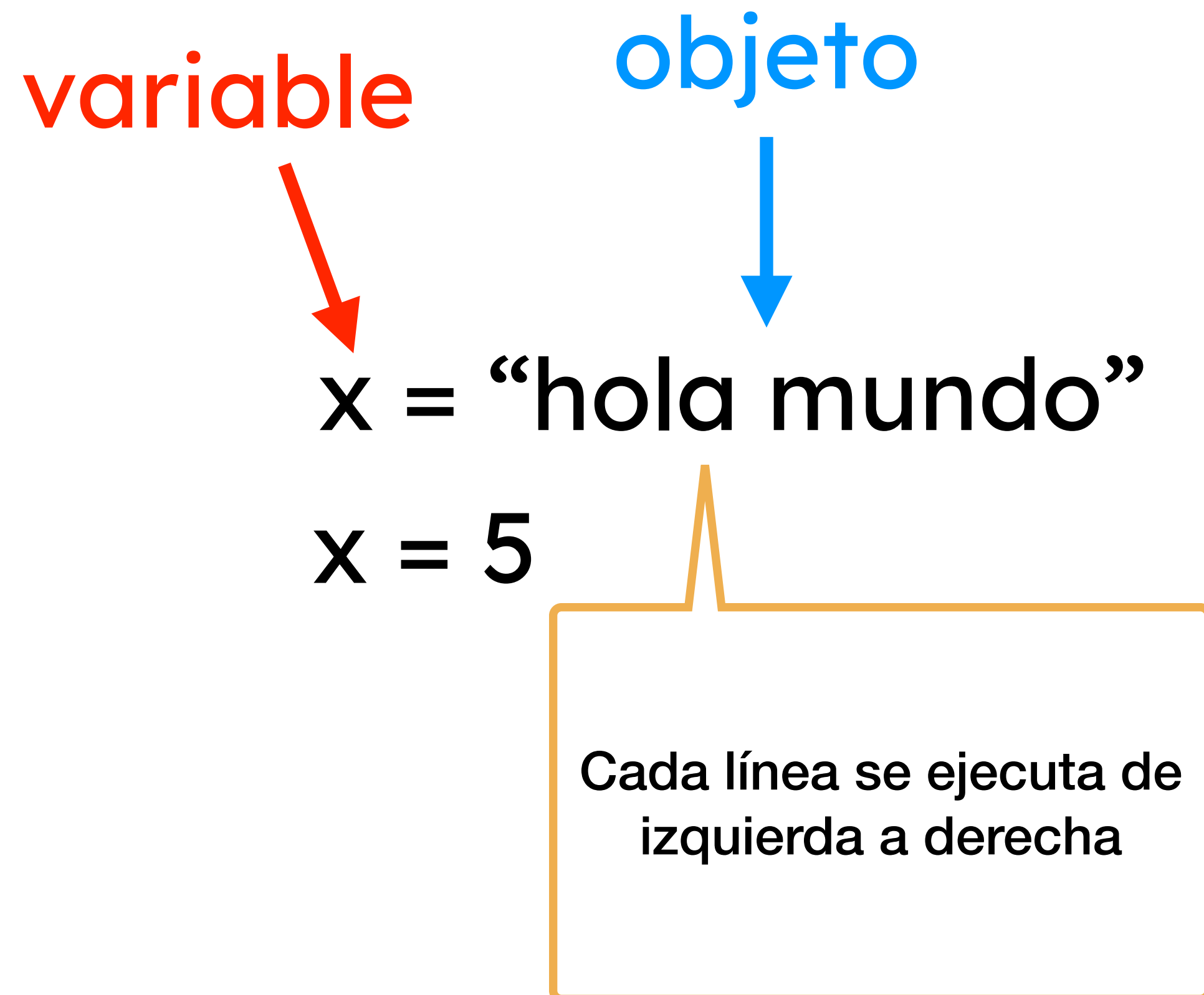
The diagram illustrates the relationship between variables and objects in Python. A red arrow points from the word 'variable' to the variable 'x' in the first two lines of code. A blue arrow points from the word 'objeto' to the string '“hola mundo”' in the first line and to the question mark in the third line.

```
x = “hola mundo”  
x = 5  
print(x)            ?
```

Memoria



Memoria

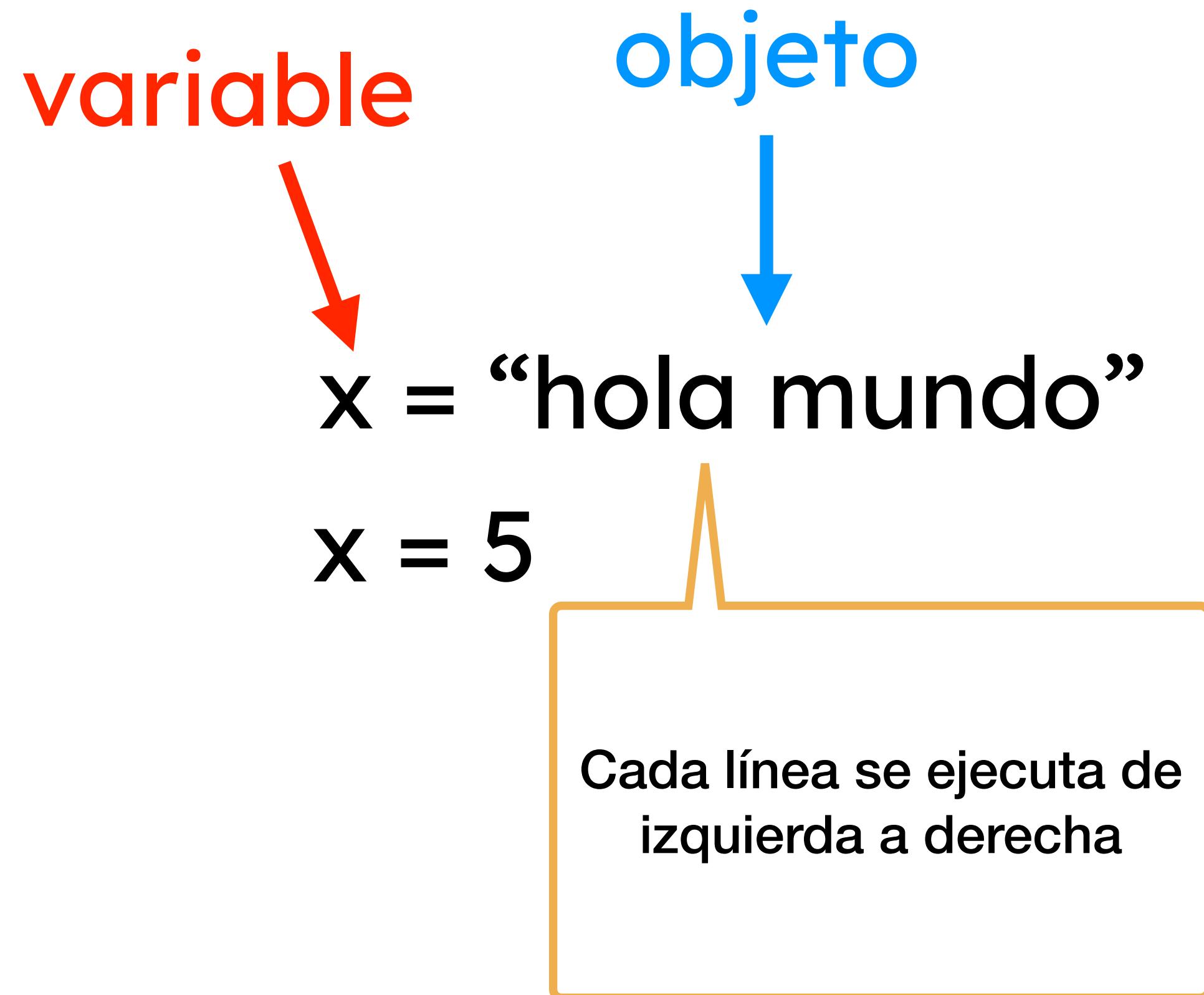


variables

objetos

"hola mundo"

Memoria



variables

objetos

x → "hola mundo"

Memoria

variable **objeto**

↓ ↓

`x = "hola mundo"`
`x = 5`

¿Qué pasa cuando se ejecuta la segunda línea?

variables

objetos

`x` → "hola mundo"

Memoria

variable **objeto**

↓ ↓

`x = "hola mundo"`
`x = 5`

¿Qué pasa cuando se ejecuta la segunda línea?

variables

objetos

x → "hola mundo"

5

Memoria

variable **objeto**

↓ ↓

`x = "hola mundo"`
`x = 5`

¿Qué pasa cuando se
ejecuta la segunda
línea?

variables

objetos

x → "hola mundo"

5

Memoria

variable **objeto**

↓ ↓

`x = "hola mundo"`
`x = 5`

¿Qué pasa cuando se ejecuta la segunda línea?

variables

objetos

x

"hola mundo"

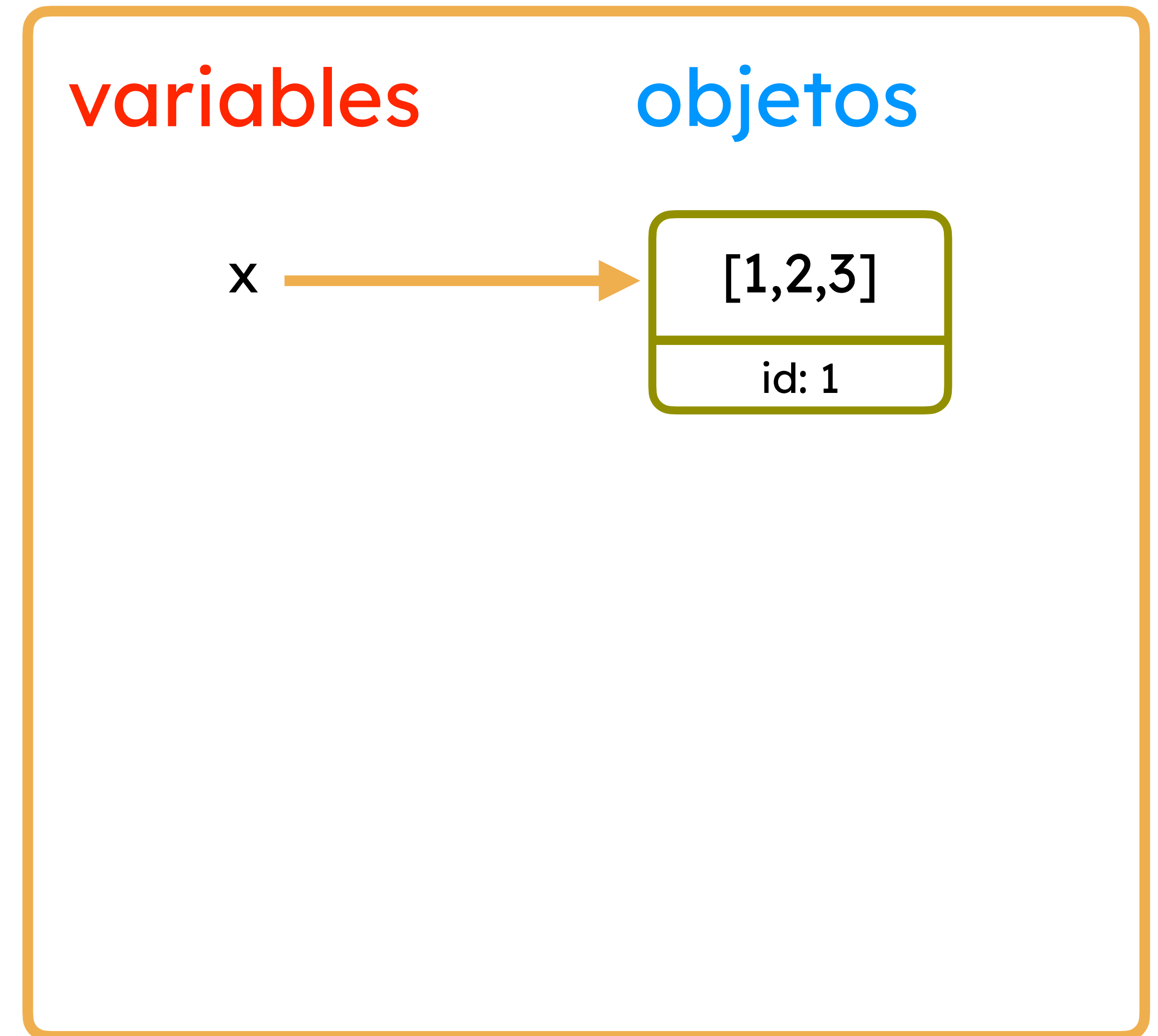
5

Memoria

variable **objeto**

↓ ↓

x = [1,2,3]



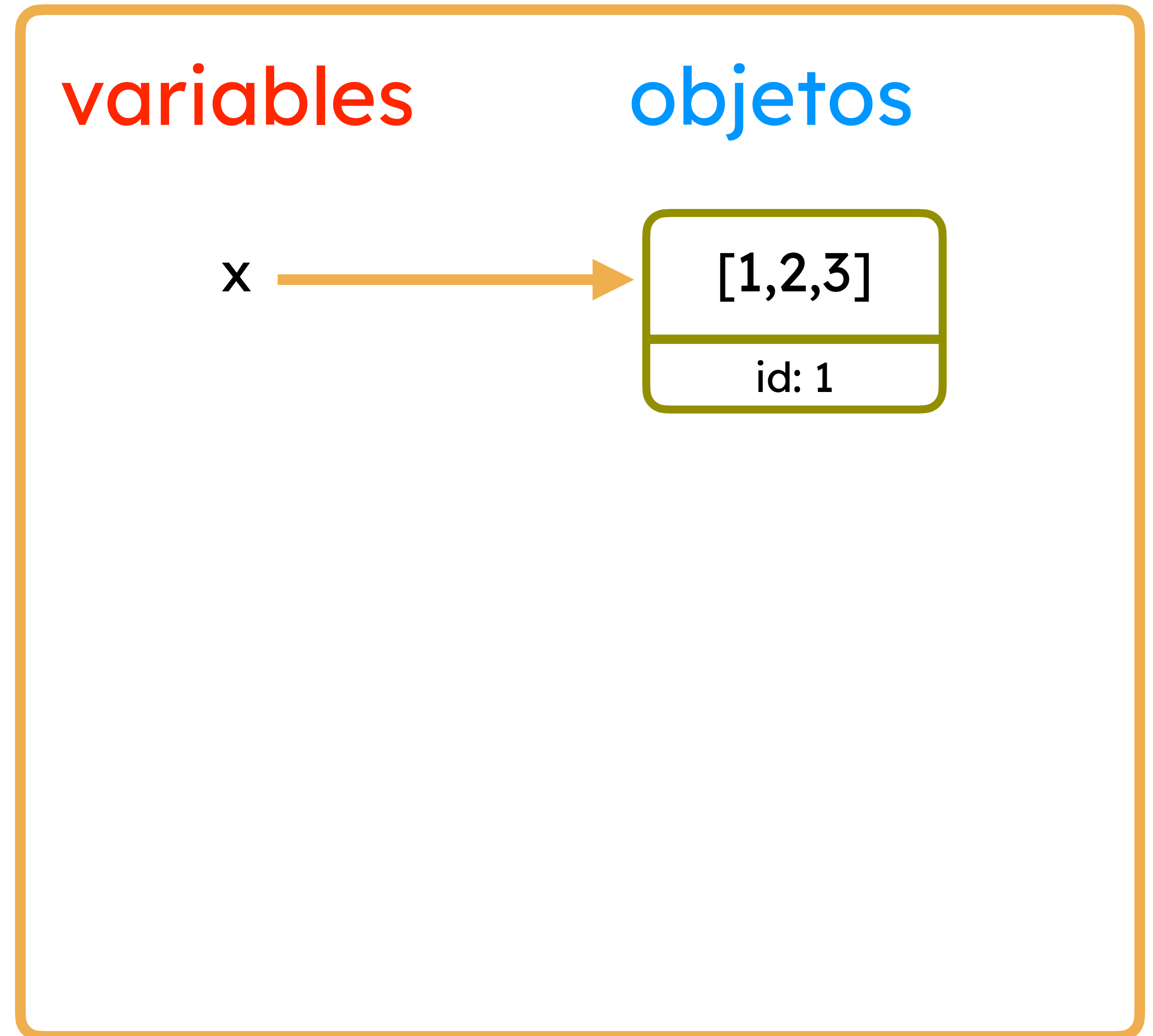
Memoria

variable **objeto**

↓ ↓

`x = [1,2,3]`

`x[0]=20`



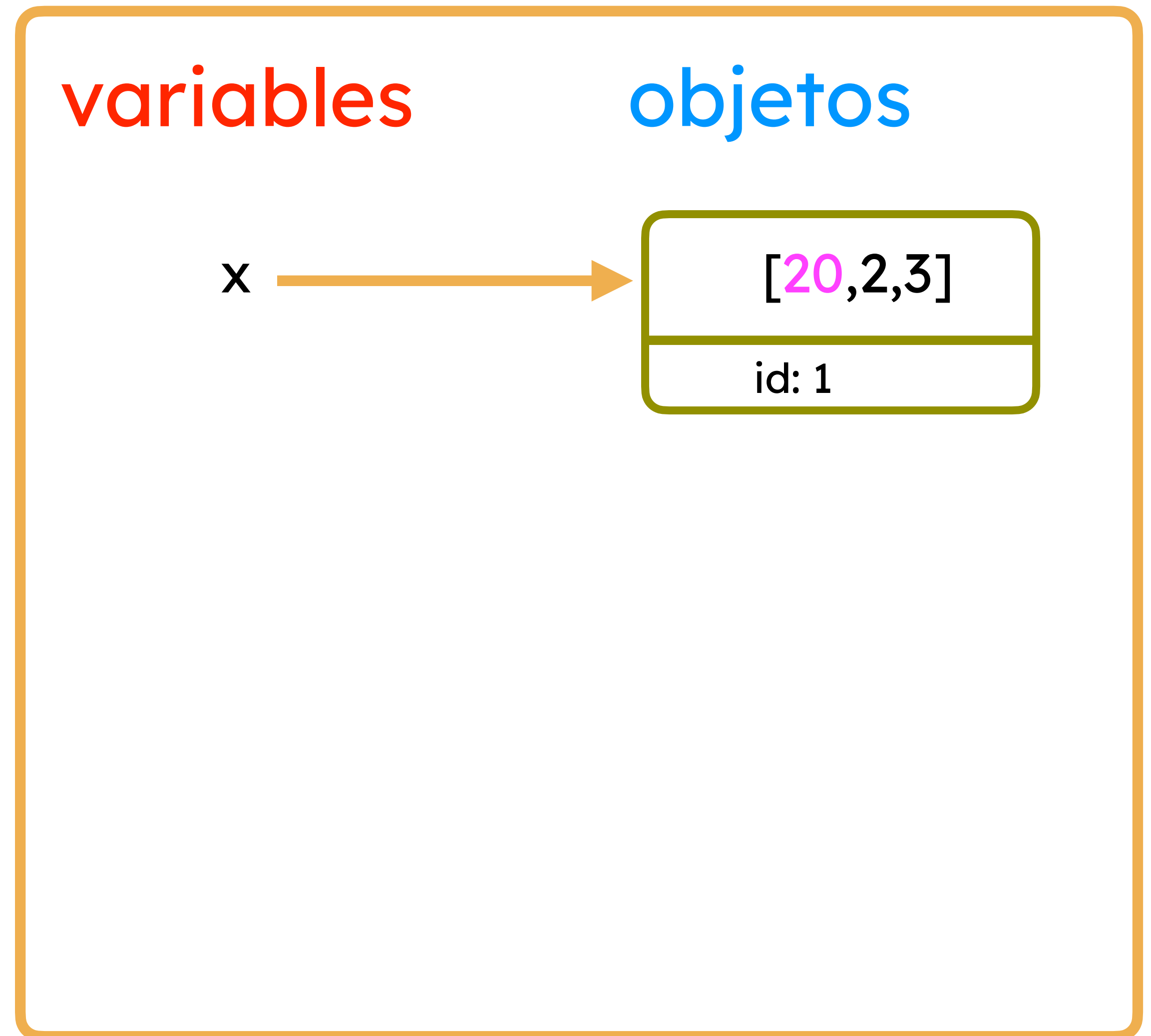
Memoria

variable **objeto**

↓ ↓

`x = [1,2,3]`

`x[0]=20`



Memoria

variable objeto



`x = [1,2,3]`

`x[0]=20`

`x[0] = [4,3]`

variables

objetos

x



[20,2,3]

id: 1

Memoria

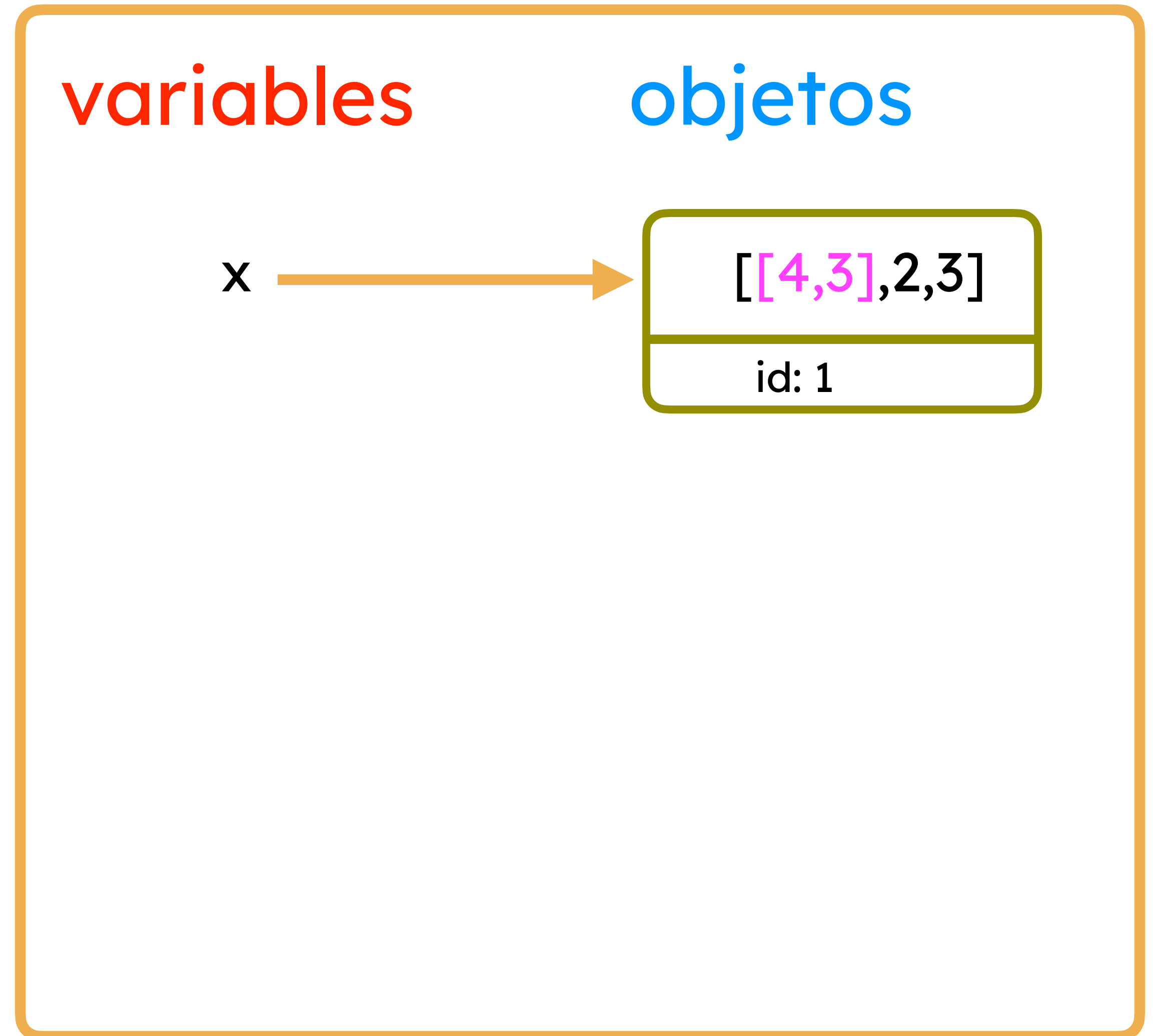
variable **objeto**

↓ ↓

`x = [1,2,3]`

`x[0]=20`

`x[0] = [4,3]`



Memoria

variable objeto



```
x = [1,2,3]
```



```
x[0]=20
```

```
x[0] = [4,3]
```

```
x[0][0] = 'hola'
```

variables

objetos

x



[4,3],2,3]

id: 1

Memoria

variable objeto



```
x = [1,2,3]
```



```
x[0]=20
```

```
x[0] = [4,3]
```

```
x[0][0] = 'hola'
```

variables

objetos

x



[['hola',3],2,3]

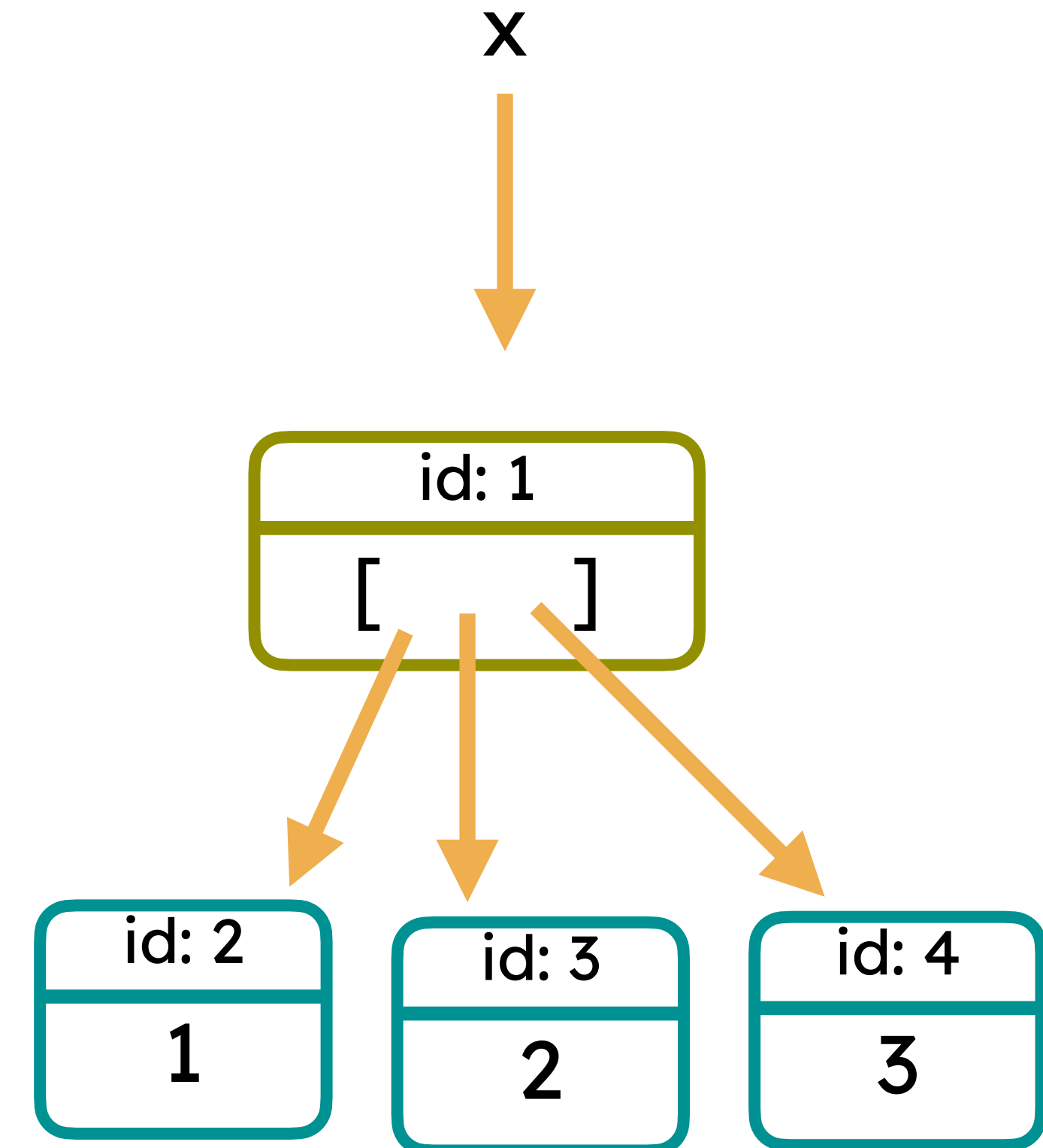
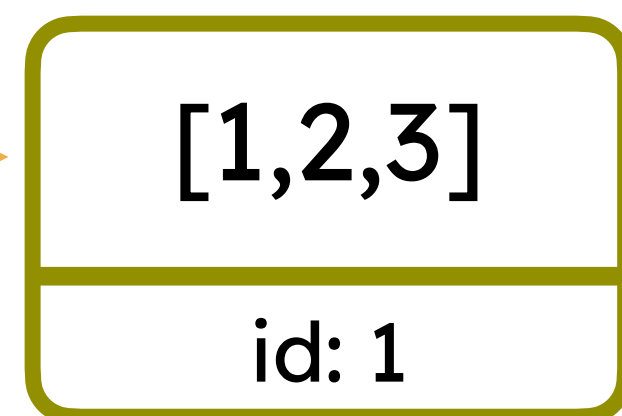
id: 1

x = [1,2,3]

variables

objetos

x



`x = [1,2,3]`

`x[0]=20`

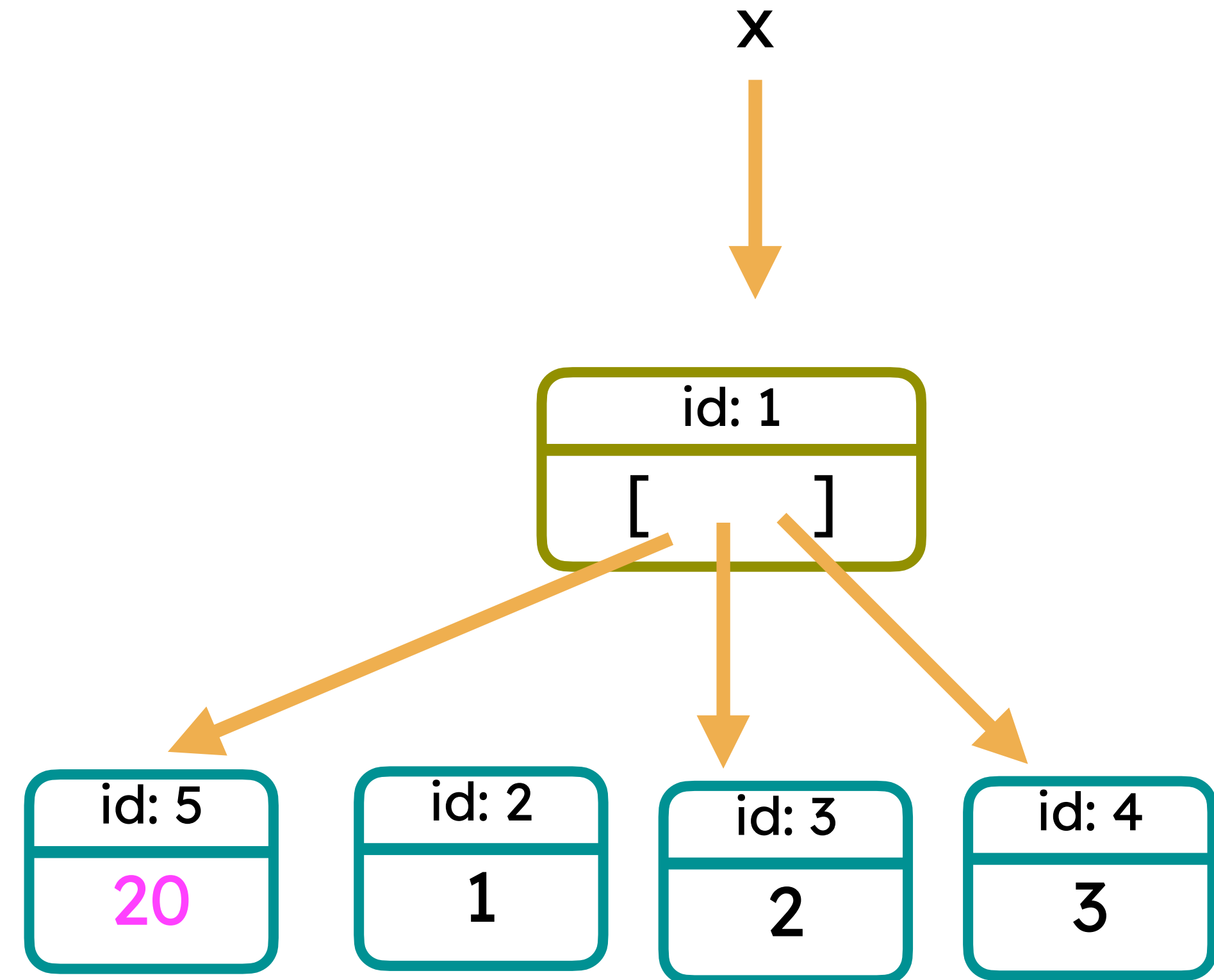
variables

objetos

x

[20,2,3]

id: 1



`x = [1,2,3]`

`x[0] = [4,3]`

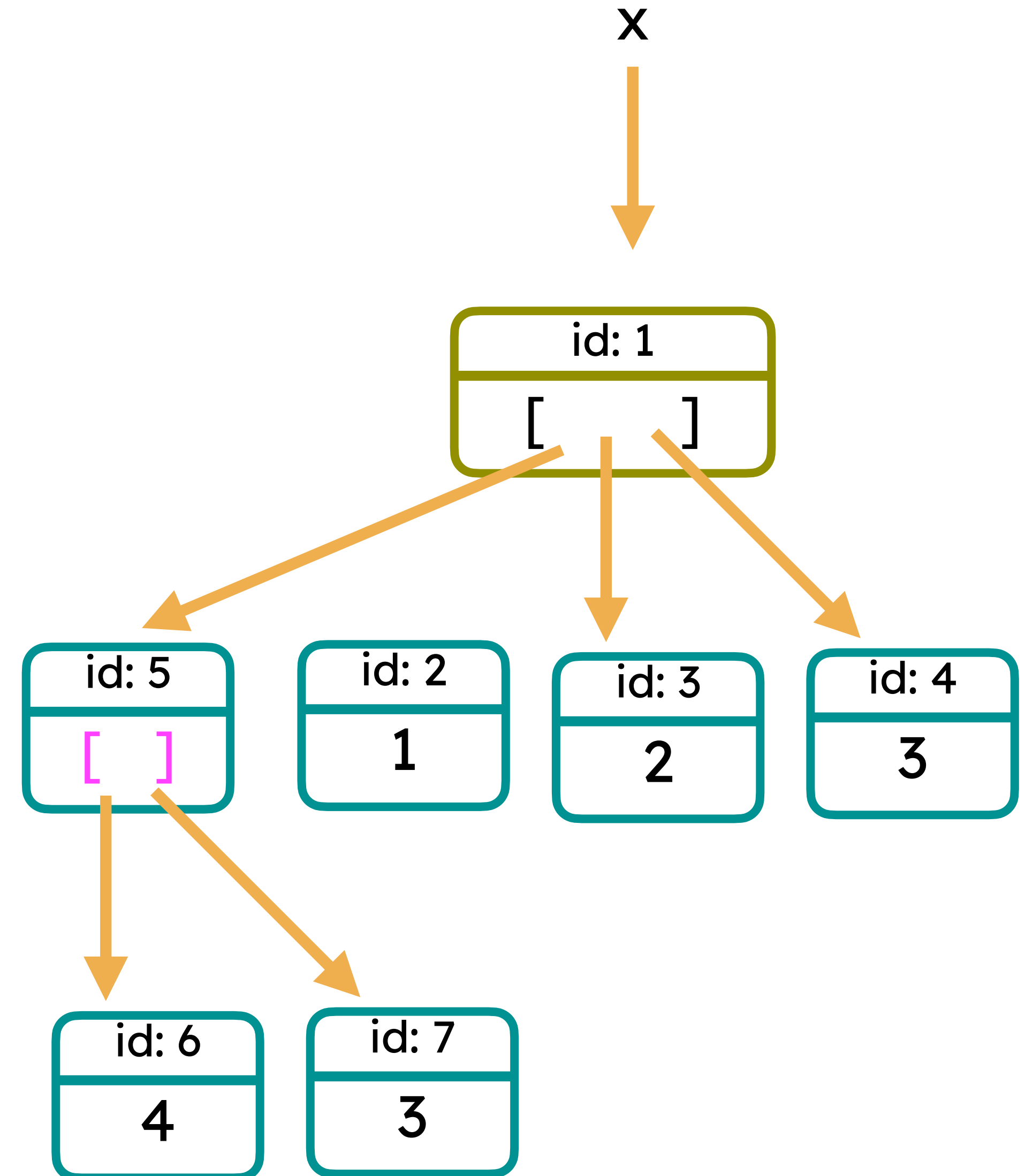
variables

objetos

x

`[[4,3],2,3]`

id: 1



`x = [1,2,3]`

`x[0]= [4,3]`

`x[0][0] = 'hola'`

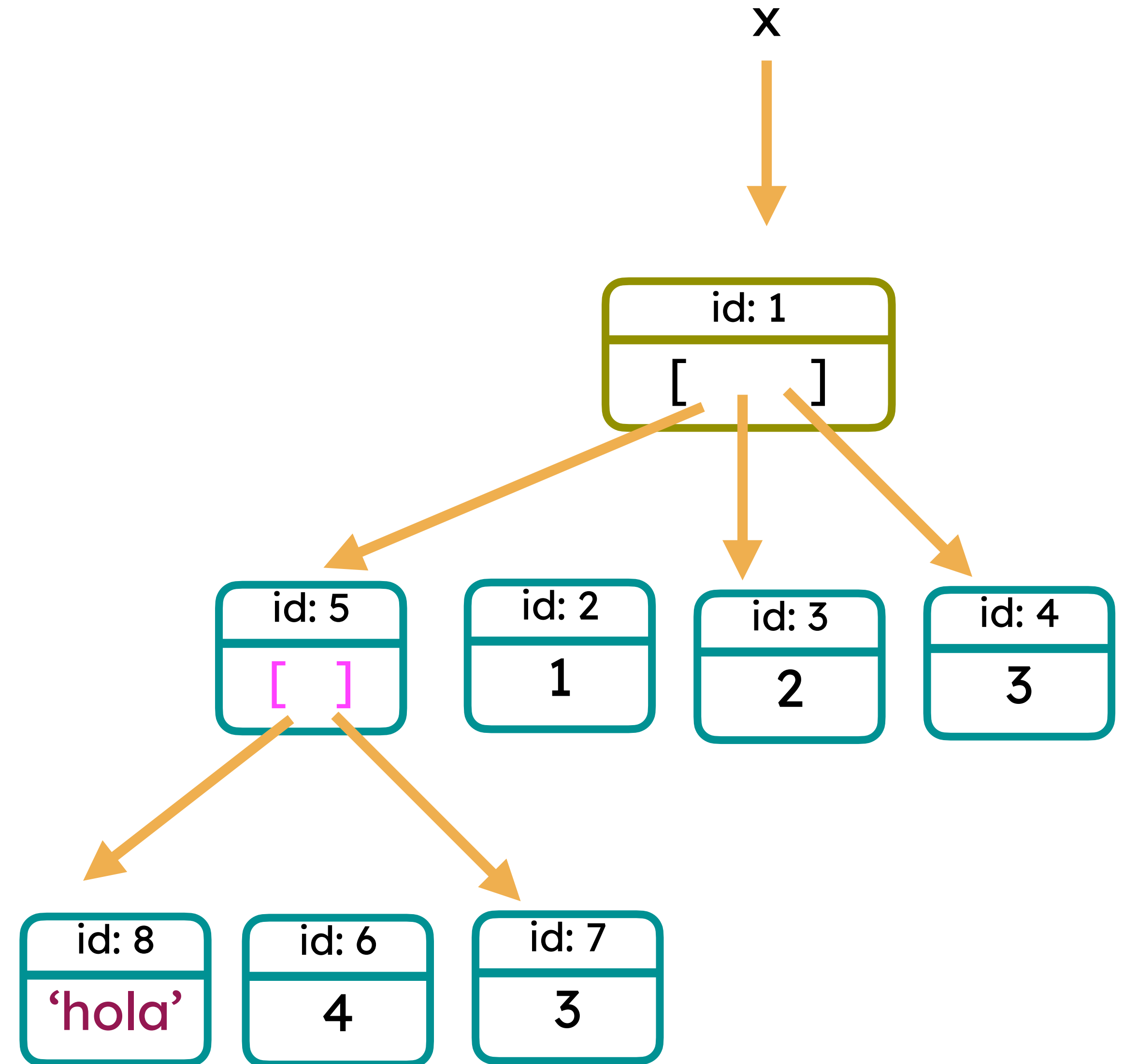
variables

objetos

x

`[['hola',3],2,3]`

id: 1



Memoria

variable **objeto**



a = "hola mundo"

b = **a**

a = (1,2)

variables

objetos



a "hola mundo"

b (1,2)

The diagram shows two variables, 'a' and 'b', on the left. Two orange arrows originate from them: one from 'a' pointing to the object '“hola mundo”' and one from 'b' pointing to the object '(1,2)'. The arrows cross each other.

Memoria

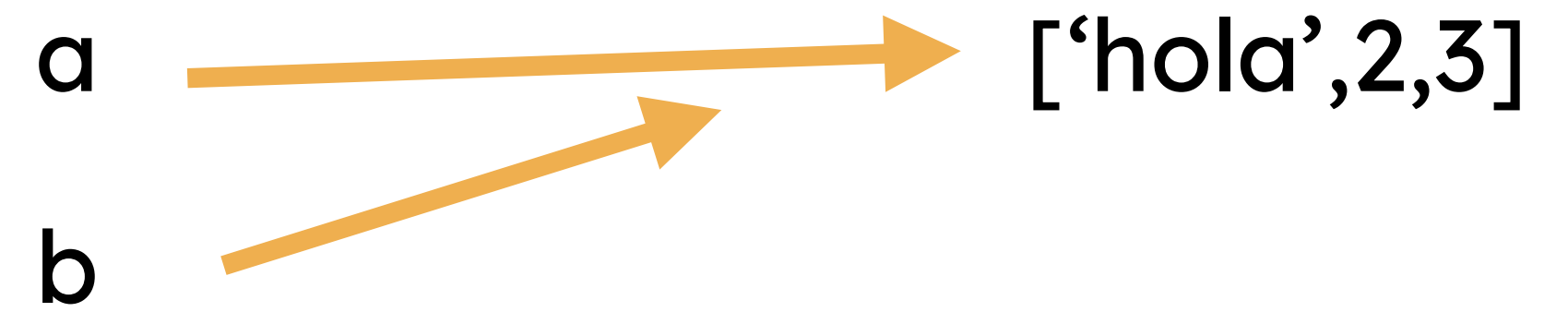
```
a = [1,2,3]
```

```
b = a
```

```
a[0] = 'hola'
```

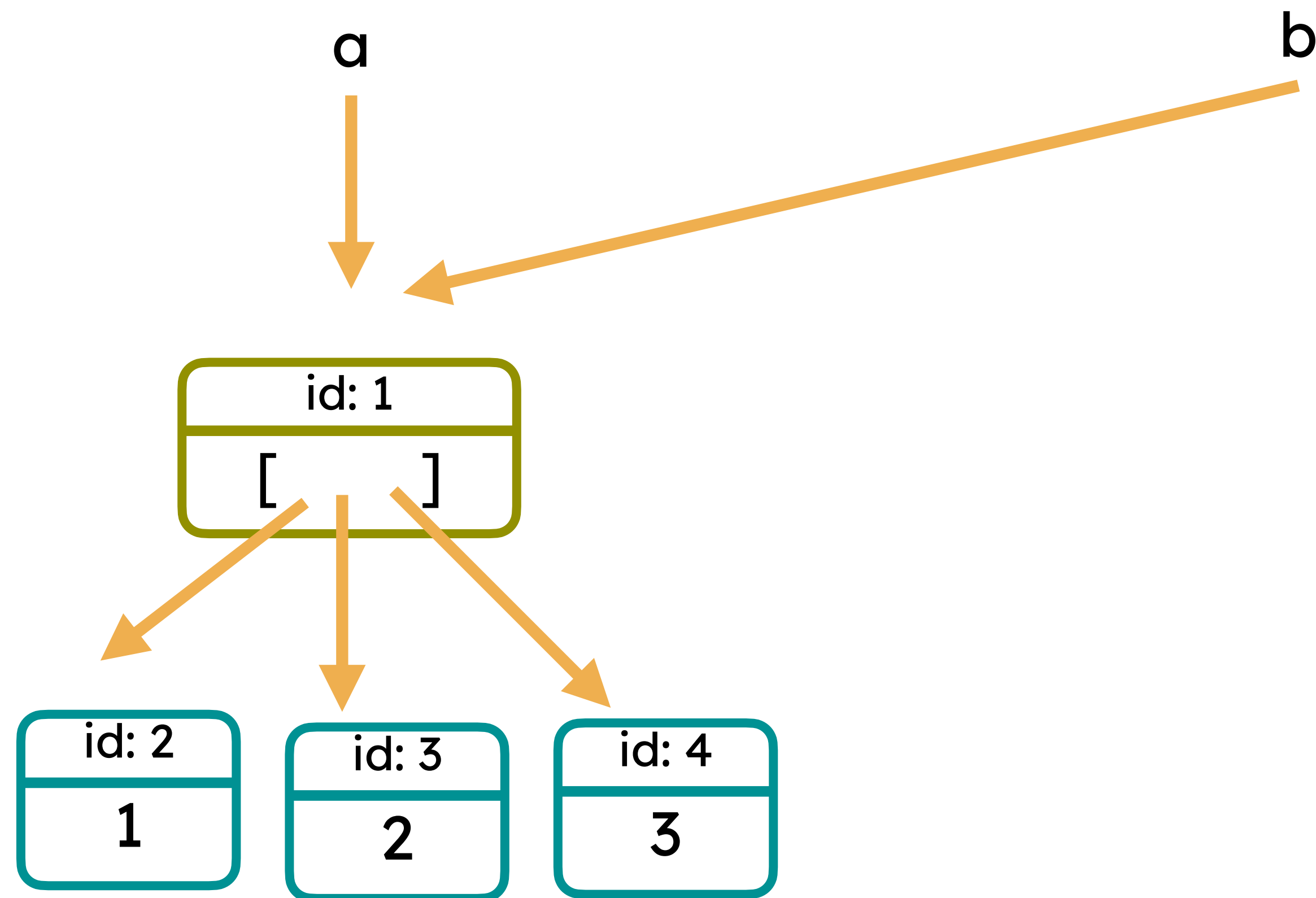
variables

objetos



`a = [1,2,3]`

`b = a`

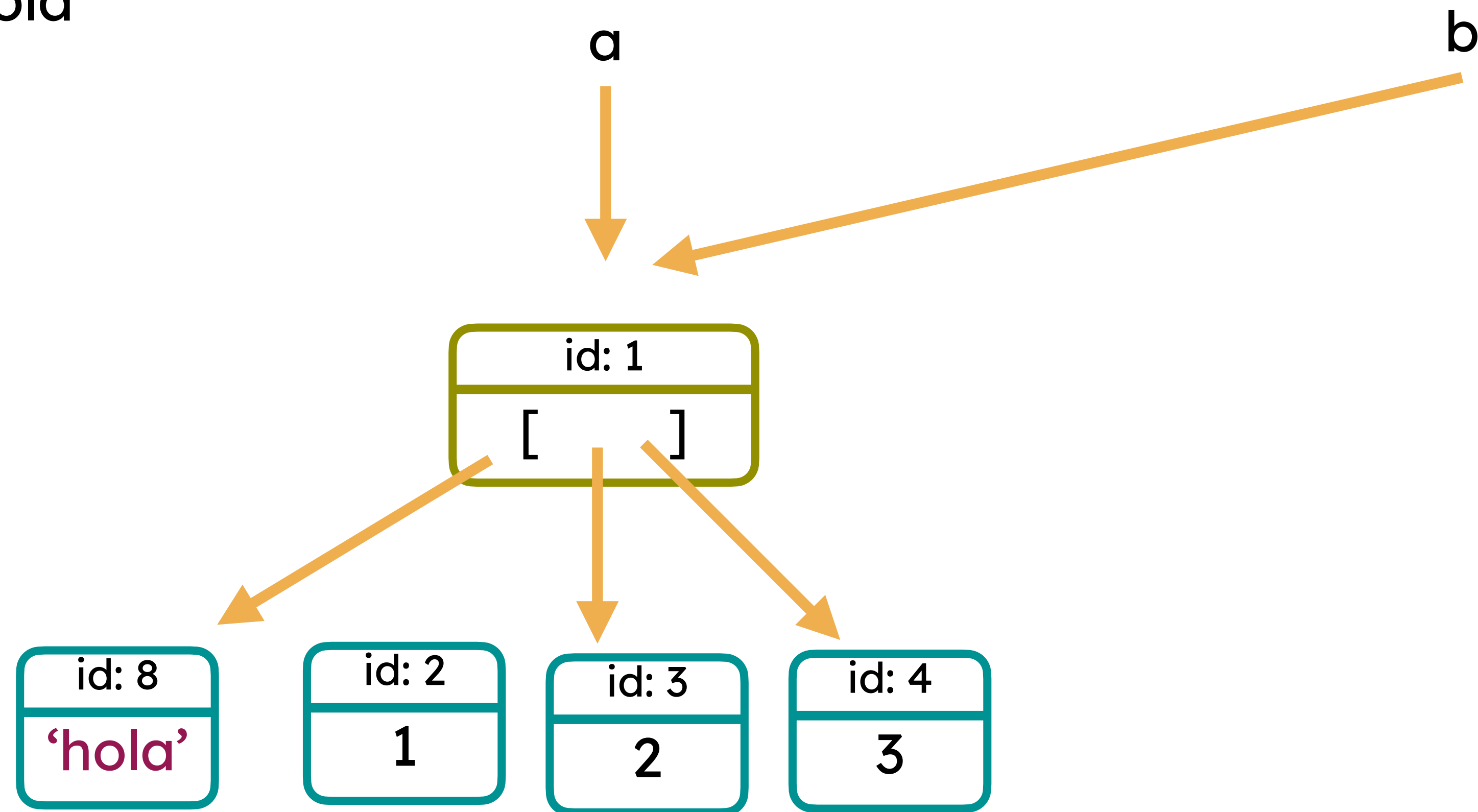


```
import copy
```

```
a = [1,2,3]
```

```
b = a
```

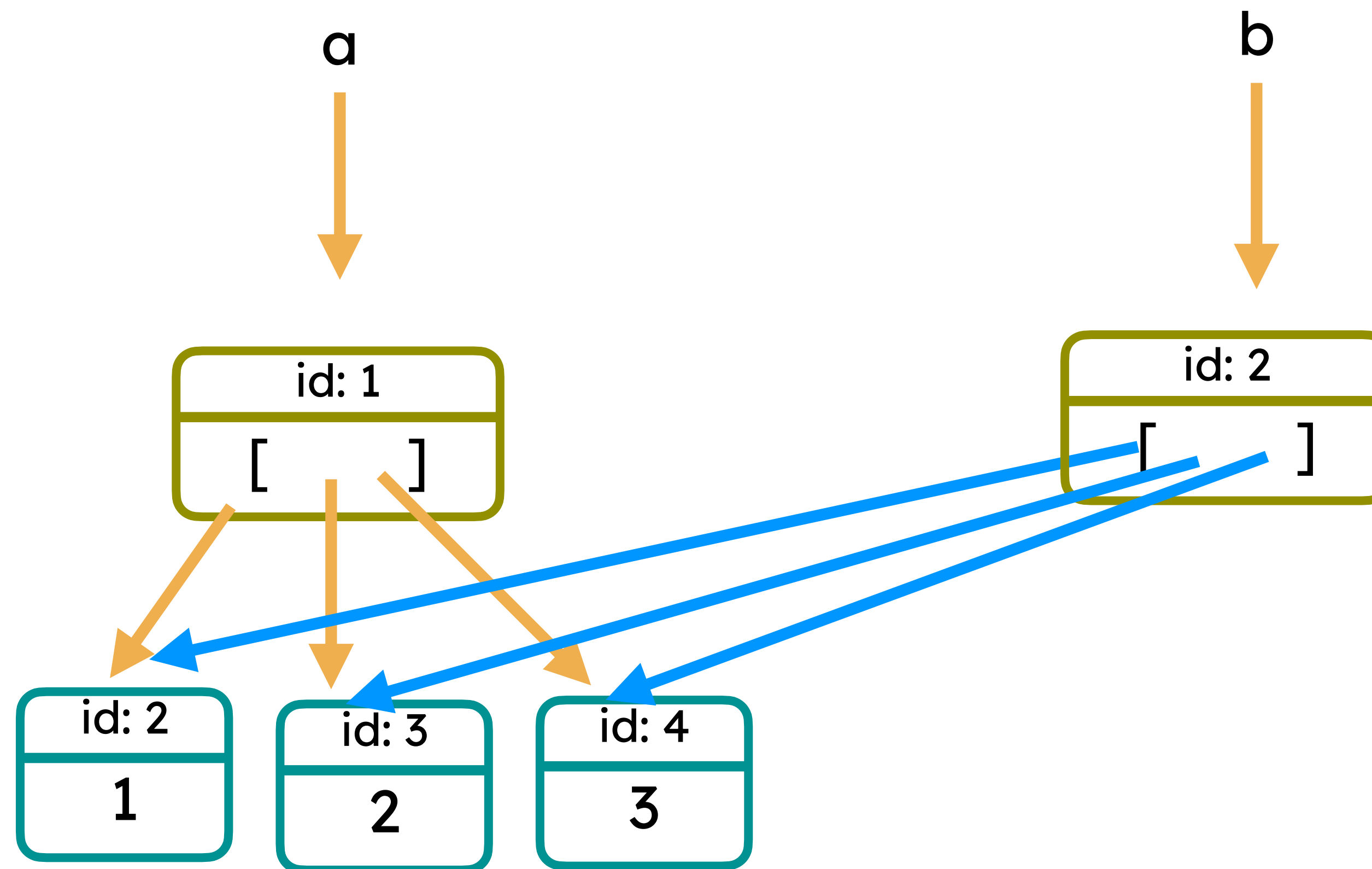
```
a[0] = 'hola'
```



```
import copy
```

```
a = [1,2,3]
```

```
b = copy.copy(a)
```

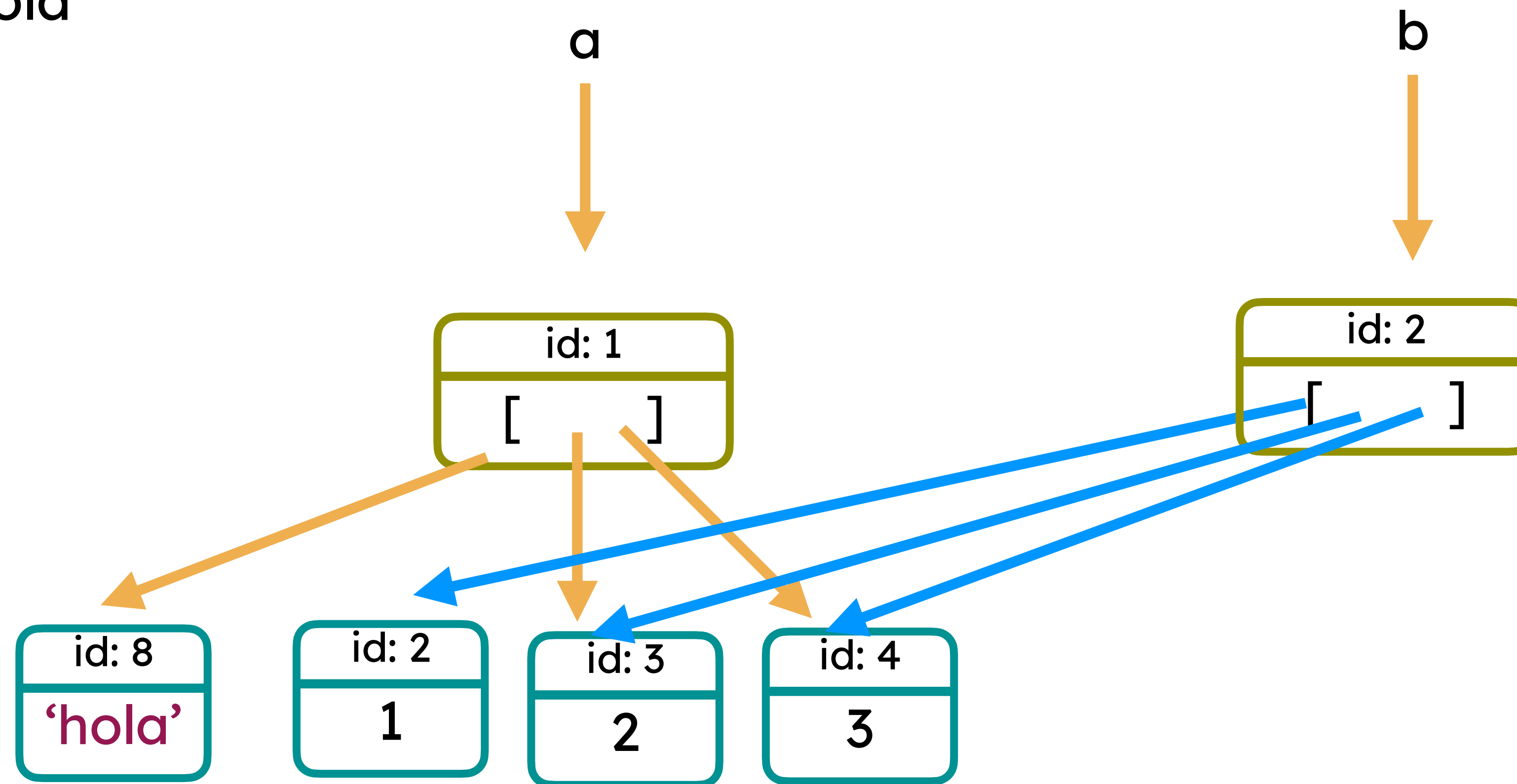


```
import copy
```

```
a = [1,2,3]
```

```
b = copy.copy(a)
```

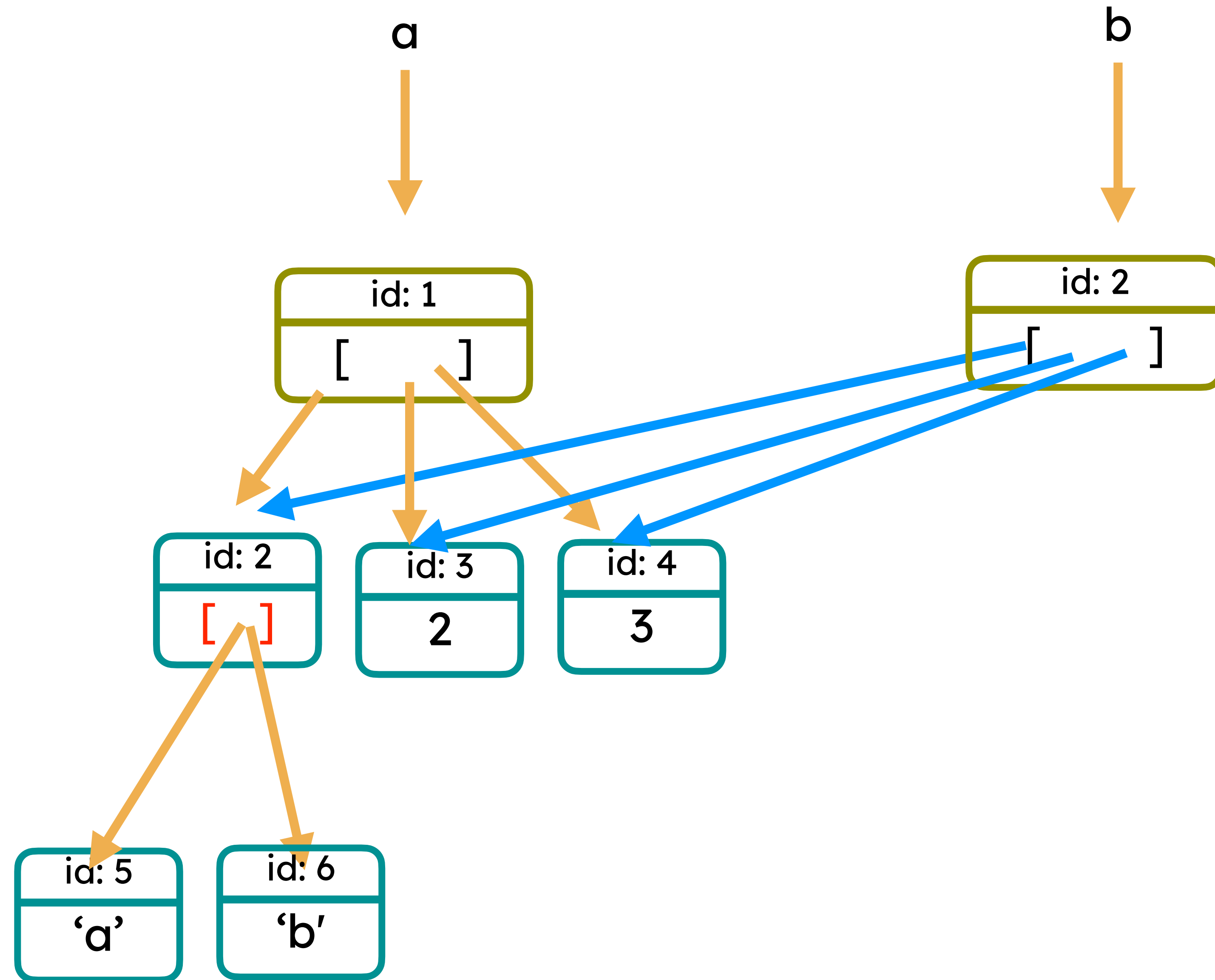
```
a[0] = 'hola'
```



```
import copy
```

```
a = [['a','b'],2,3]
```

```
b = copy.copy(a)
```

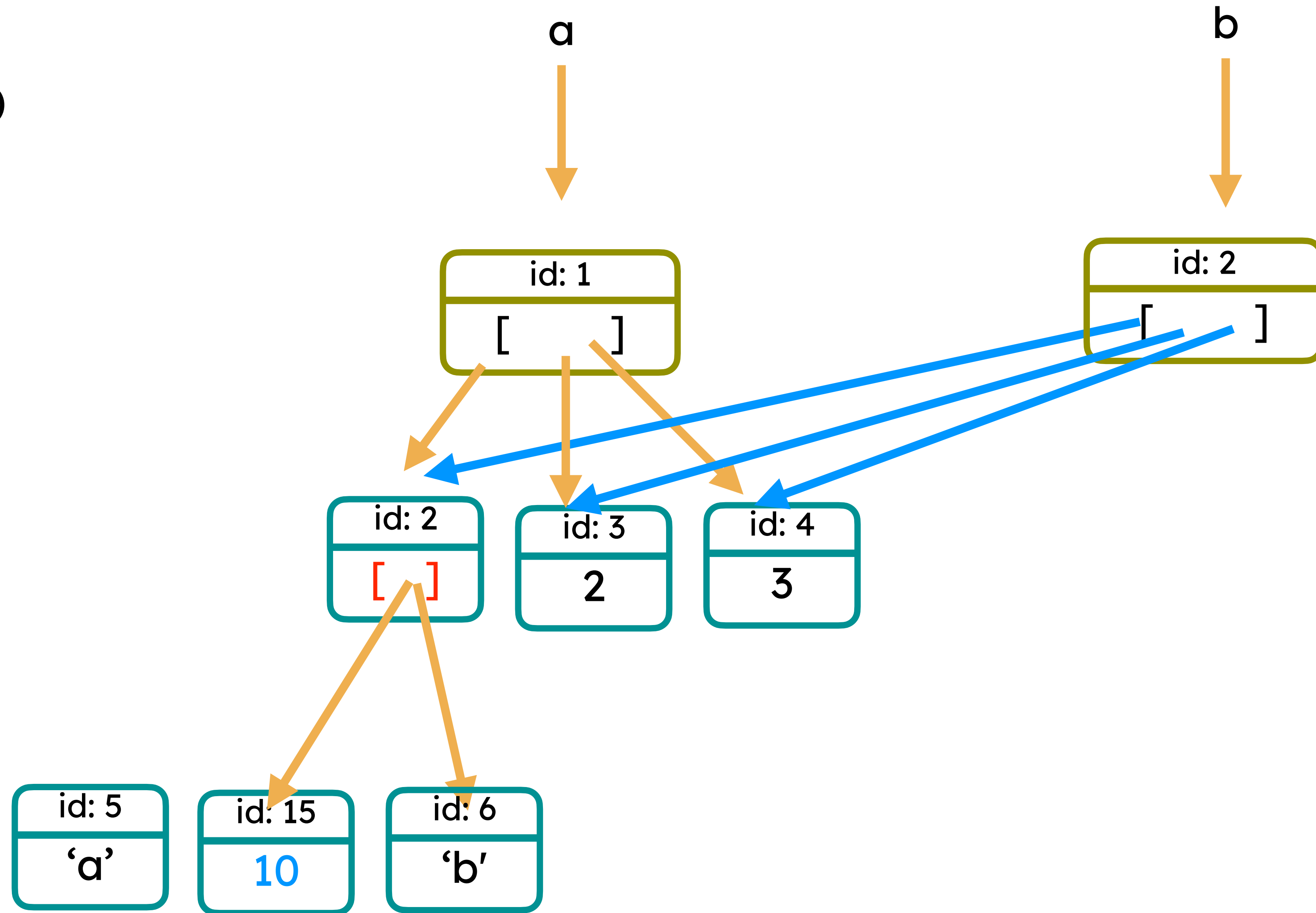


```
import copy
```

```
a = [['a','b'],2,3]
```

```
b = copy.copy(a)
```

```
a[0][0] = 10
```

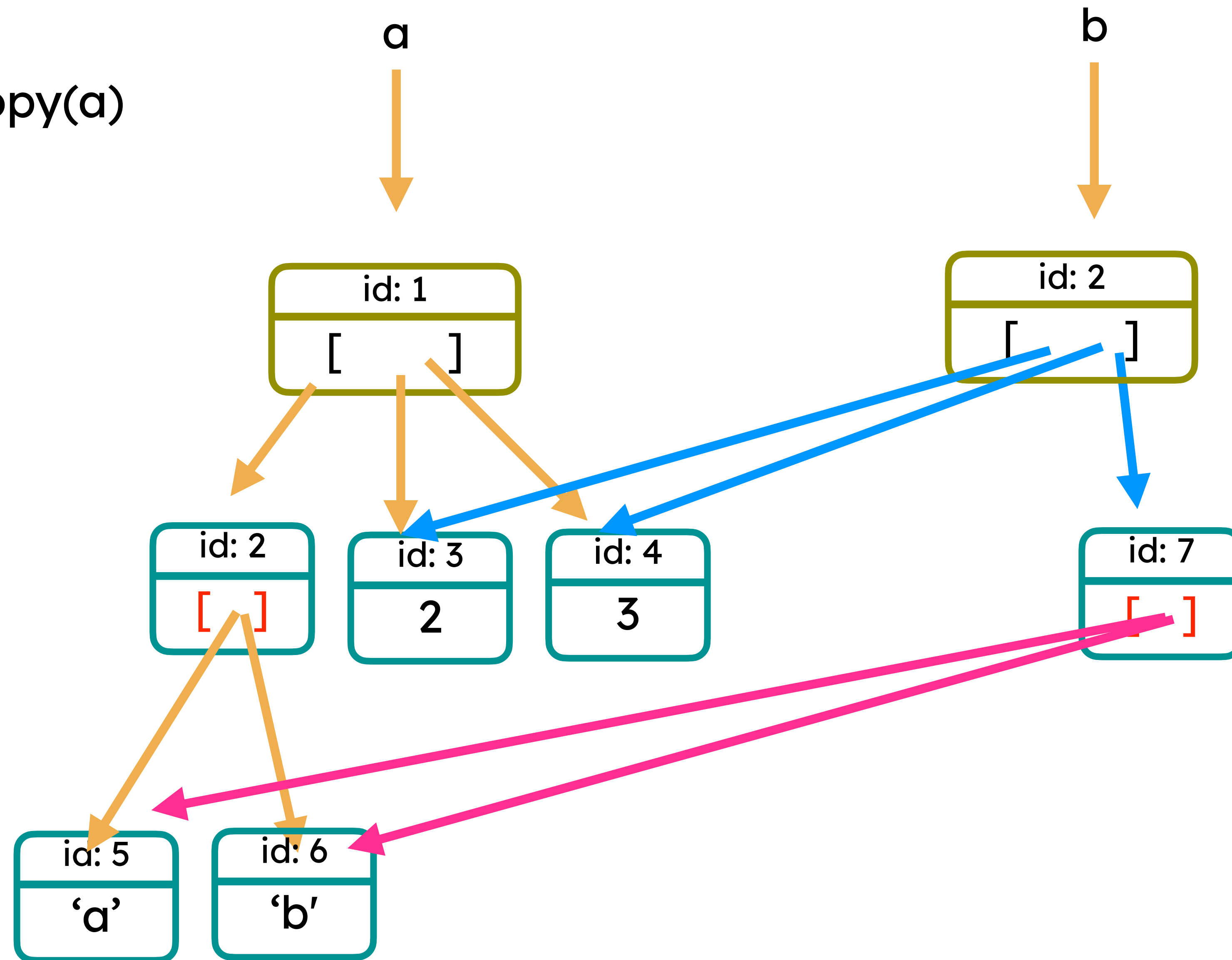


```
import copy
```

```
a = [['a','b'],2,3]
```

```
b = copy.deepcopy(a)
```

```
a[0][0] = 10
```

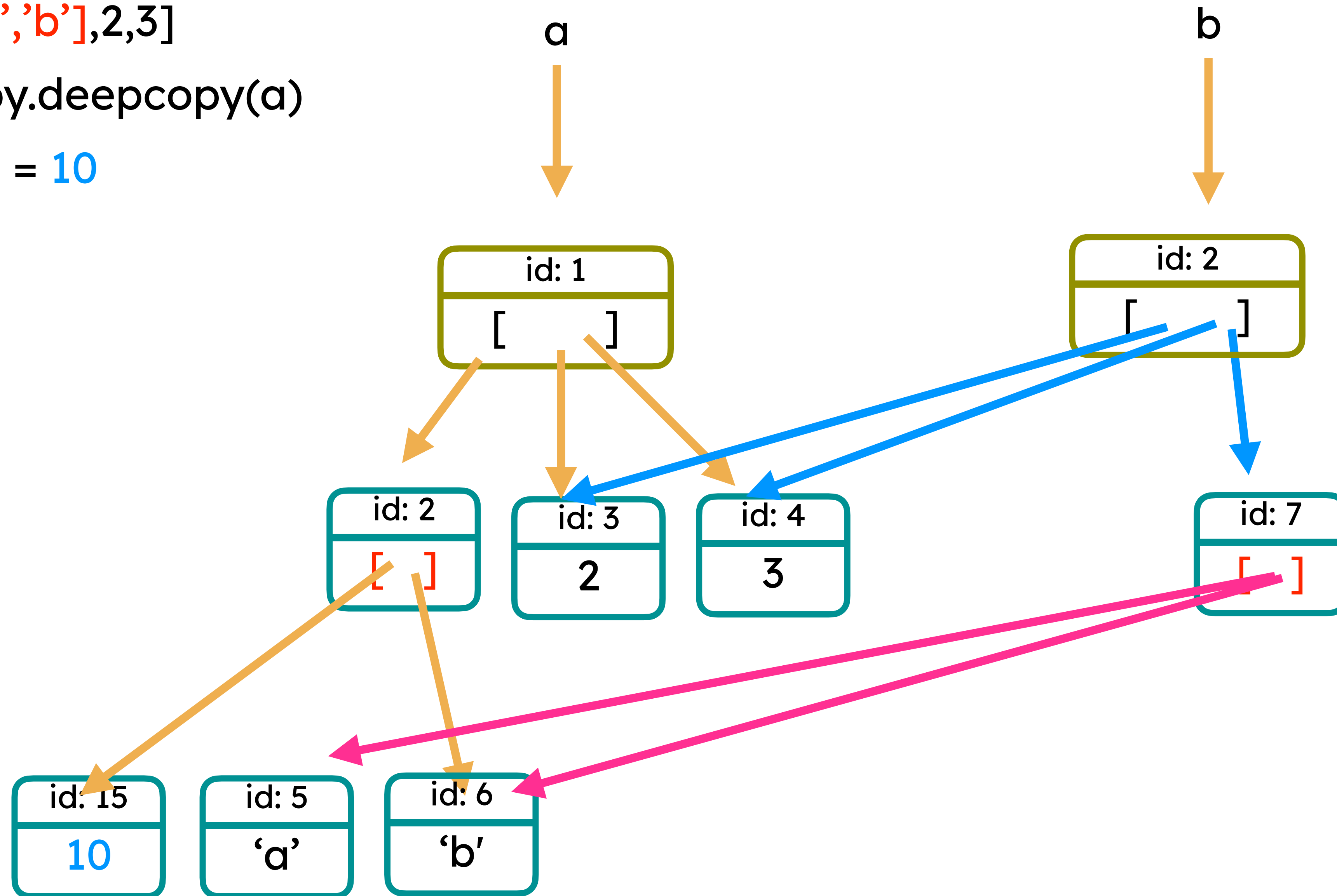


```
import copy
```

```
a = [['a','b'],2,3]
```

```
b = copy.deepcopy(a)
```

```
a[0][0] = 10
```



Listas vs tuplas

mutables

Listas

[1,2,3]

inmutables

Tuplas

(1,2,3) No podemos modificar estos objetos

¿Qué pasa si tenemos listas anidados?

[1,2,[3,4,5]]

(1,2,[3,4,5])

¿Puedo modificar la lista anidada dentro de la tupla?

Listas vs tuplas

mutables

Listas

[1,2,3]

inmutables

Tuplas

(1,2,3) No podemos modificar estos objetos

¿Qué pasa si tenemos listas anidados?

[1,2,[3,4,5]]

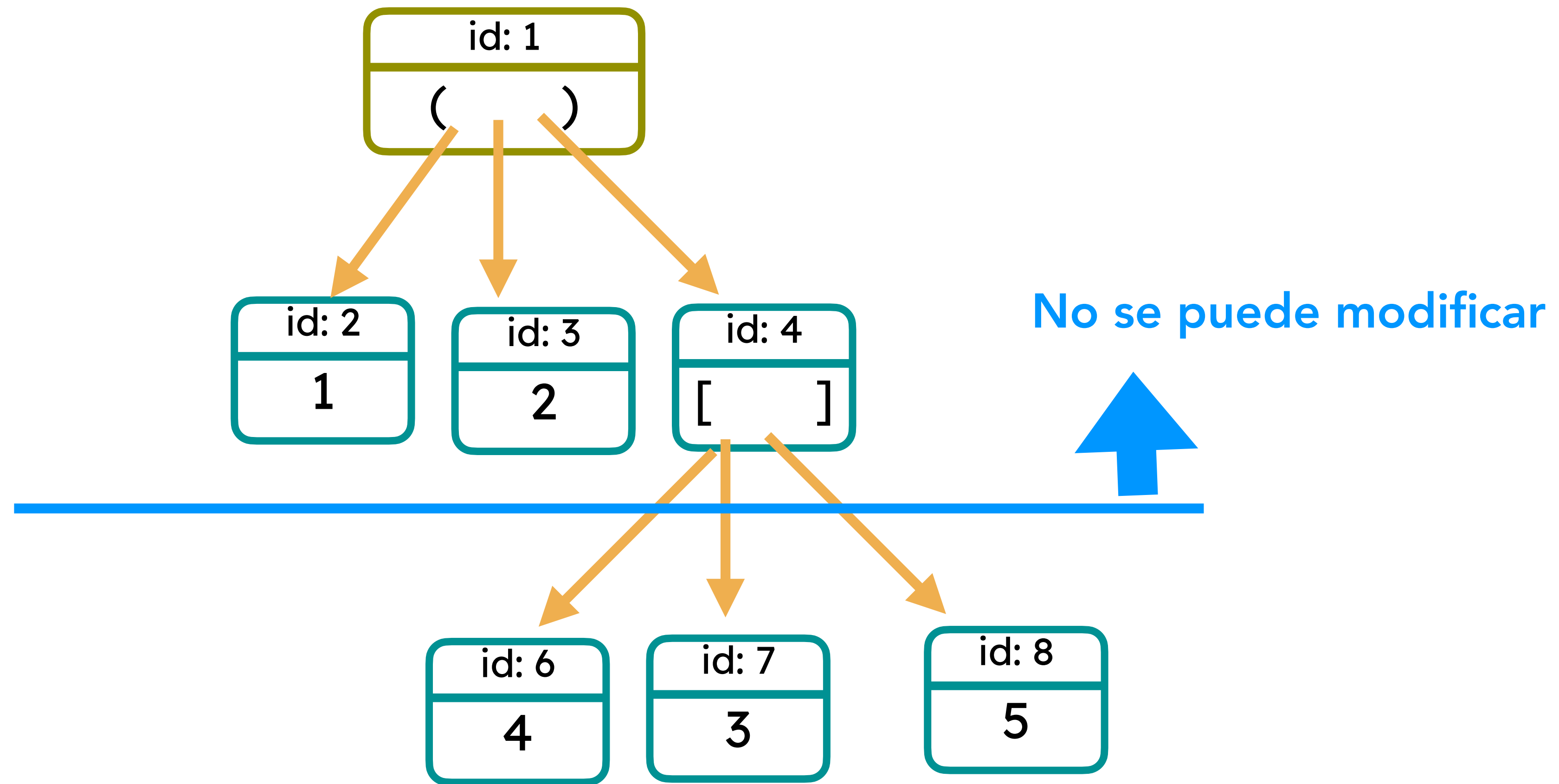
(1,2,[3,4,5])

¿Puedo modificar la lista anidada dentro de la tupla?

Podemos modificar los elementos dentro de la lista (porque es mutable)
pero no podemos cambiar el tipo del tercer elemento.

Siempre el tercer elemento es una lista porque la tupla es inmutable.

(1,2,[3,'hola',5])



Inmutables

cadenas

```
c = "hola mundo"
s = str(123)
```

tuplas

```
t = (1, 2, 3)
t = (1,)
t = 1, 2, 3
t = tuple("abc")
```

frozenset

```
f1 = frozenset()
f2 = frozenset([1, 2, 3])
f3 = frozenset("abc")
```

* también podemos crear listas, sets, diccionarios,... por comprensión

mutables

listas

```
l = [1, 2, 3]
l = [1]
l = list("abc")
```

sets

```
s1 = {1, 2, 3}
s2 = set()
s3 = set([1, 2, 3])
s4 = set("abc")
```

diccionarios

```
d1 = {}
d2 = {"a": 1, "b": 2}
d3 = dict()
d4 = dict(a=1, b=2)
d5 = dict([("a", 1), ("b", 2)])
claves = ["a", "b"]
valores = [1, 2]

d6 = dict(zip(claves, valores))
d7 = dict.fromkeys(claves, 0)
```

**¿Cómo accedemos a los elementos
de una colección?**

**¿Cómo modificamos los
elementos de una colección?**

Palabras reservadas de python

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',  
 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',  
 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',  
 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try',  
 'while', 'with', 'yield']
```

f-string

- f-string (formatted string literal) es una forma de insertar valores dentro de un string. Se pone una f antes de las comillas y las expresiones van entre {}.
- Todo lo que va entre {} se evalúa y se convierte a string automáticamente.

`f"texto {expresión:formato} más texto"`

f-string

- Pf-string (formatted string literal) es una forma de insertar valores dentro de un string. Se pone una f antes de las comillas y las expresiones van entre {}.
- Todo lo que va entre {} se evalúa y se convierte a string automáticamente.

`f"texto {expresión:formato} más texto"`

```
x = 10
y = 20
print(f"{x=}, {y=}")          # "x=10, y=20"
print(f"{x + y = }")          # "x + y = 30"
print(f"{nombre.upper() = }") # "nombre.upper() = 'ANA'"
```

```
n = 3.14159

f"{n:.2f}"    # "3.14" (2 decimales)
f"{n:.4f}"    # "3.1416" (4 decimales)

grande = 1000000
f"{grande:,}" # "1,000,000" (separador de miles)
f"{grande:_}" # "1_000_000" (separador con guion bajo)

# Porcentaje
ratio = 0.75
f"{ratio:.1%}" # "75.0%"
```

```
texto = "hola"

f"{texto:>10}" # "      hola" (derecha, 10 chars)
f"{texto:<10}" # "hola      " (izquierda)
f"{texto:^10}" # "    hola    " (centrado)
f"{texto:*^10}" # "***hola***" (relleno con *)

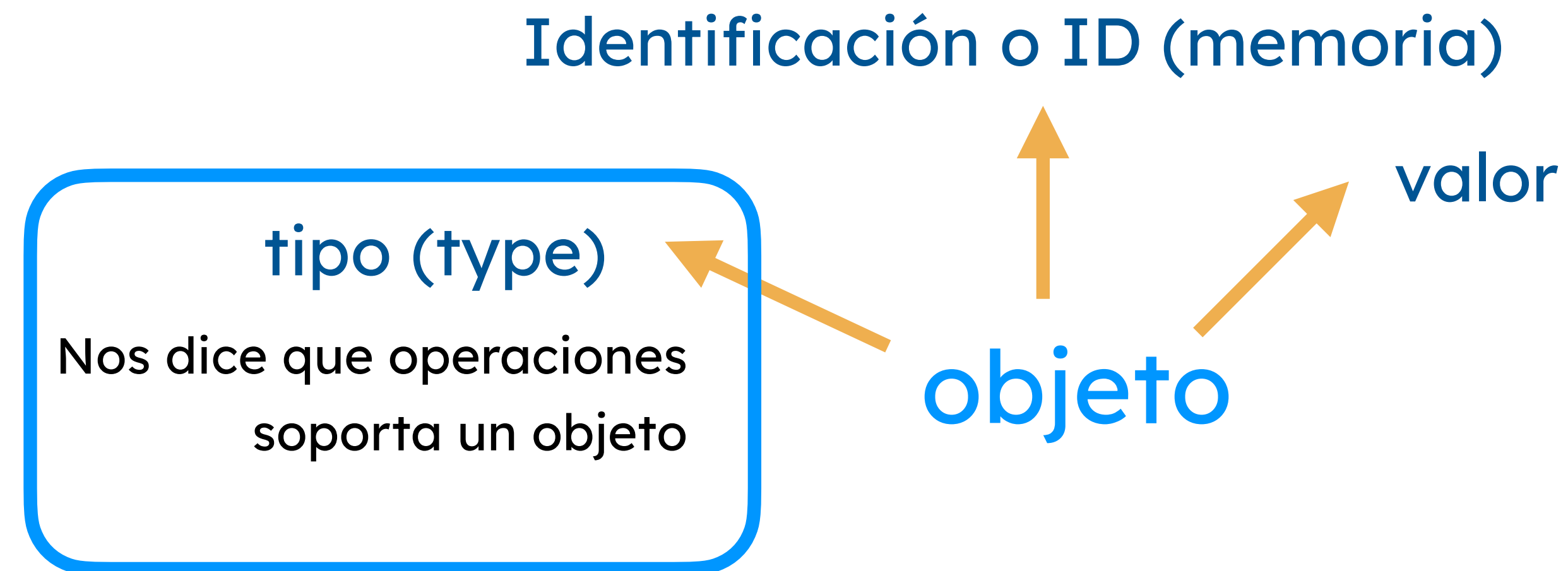
num = 42
f"{num:05}"    # "00042" (padding con ceros)
```

```
# expresiones dentro
nombre = 'Ana'
f"{2 + 2}"          # "4"
f"{nombre.upper()}" # "ANA"
f"{'par' if x % 2 == 0 else 'impar'}" # "par"
f"{[i**2 for i in range(3)]}" # "[0, 1, 4]"

#bases numericas
n = 255

f"{n:b}"    # "11111111" (binario)
f"{n:o}"    # "377" (octal)
f"{n:x}"    # "ff" (hexadecimal minúscula)
f"{n:X}"    # "FF" (hexadecimal mayúscula)
```

Variables y objetos



Operadores y polimorfismo

tipo (type)

Nos dice que operaciones
soporta un objeto

- Los operadores son los símbolos o palabras clave que se usan para hacer operaciones con objetos

Operadores aritméticos

Operador	Números	Strings	Listas	Tuplas	Sets	Diccionarios
+	Suma	Concatena	Concatena	Concatena	X	X
-	resta	X	X	X	Diferencia	X
*	Multiplica	Repite	Repite	Repite	X	X
/	Divide	X	X	X	X	X
//	Div entera	X	X	X	X	X
%	Módulo	X	X	X	X	X
**	Potencia	X	X	X	X	X

Operadores de comparación

Operador	Números	Strings	Listas	Tuplas	Sets	Diccionarios
==	✓	✓	✓	✓	✓	✓
!=	✓	✓	✓	✓	✓	✓
<	✓	lexicográfico	lexicográfico	lexicográfico	✓ (subconjunto estricto)	✗
>	✓	✓	✓	✓	✓ (superconjunto estricto)	✗
<=	✓	✓	✓	✓	✓ (subconjunto)	✗
>=	✓	✓	✓	✓	✓ (superconjunto)	✗
Nota: podemos usar la función ord(): recibe un carácter y devuelve su código Unicode (un número entero).						

Operadores lógicos

Operador	Cualquier objeto
and	✓
or	✓
not	✓
Nota: Nota: todo lo que no sea False, None, 0, ' ', [], {} se considera True.	

and		
Valor de a	Valor de b	Resultado
FALSE	FALSE	a (primer falsy)
FALSE	TRUE	a (el primer falsy)
TRUE	FALSE	b (segundo operando)
TRUE	TRUE	b (segundo operando)
Regla: and devuelve el primer falsy si lo hay; si no, devuelve el segundo operando.		

or		
Valor de a	Valor de b	Resultado
FALSE	FALSE	b (segundo operando)
FALSE	TRUE	b (segundo operando)
TRUE	FALSE	a (el primer truthy)
TRUE	TRUE	a (el primer truthy)
Regla: or devuelve el primer truthy si lo hay; si no, devuelve el segundo operando.		

not	
Valor de a	Resultado
FALSE	TRUE
TRUE	FALSE
not invierte la verdad del operando y siempre devuelve un booleano.	

Operadores de identidad

Operador	Cualquier objeto
is	✓
is not	✓
Nota: comparan ID	

Operadores de pertenencia

Operador	Números	Strings	Listas	Tuplas	Sets	Diccionarios
in	✓	✓	✓	✓	✓	✓ (busca en keys)
not in	✓	✓	✓	✓	✓	✓
Nota: Verifican si un valor existe dentro de un contenedor o iterable. Básicamente se pueden usar con cualquier cosa que puedas recorrer con for, puedes usar in. Ej: range, generadores, bytes, archivos (línea por línea).						

Operadores bitwise

Operador	Enteros		Sets	Diccionarios
&	AND bit a bit	Cada bit del resultado es 1 si ambos bits correspondientes son 1.	Intersección	X
	OR bit a bit	Cada bit es 1 si al menos uno es 1	Unión	Merge (3.9+)
^	XOR bit a bit	Cada bit es 1 si exactamente uno es 1	Diferencia simétrica	X
~	NOT bit a bit	invierte todos los bits (complemento a uno)	X	X
<<	Shift izquierda	Mueve bits n posiciones a la izquierda, rellena con ceros	X	X
>>	Shift derecha	Mueve bits n posiciones a la derecha	X	X
Nota: Funcionan a nivel de bits individuales de los números enteros. Tienen sus propios operadores de asignación aumentada. Usar la función bin() y int(): print(int(0b1000000))				

Operadores de asignación aumentada

Operador	Números	Strings	Listas	Tuplas	Sets	Diccionarios
<code>+=</code>	✓	✓	✓	✓	X	X
<code>-=</code>	✓	X	X	X	✓	X
<code>*=</code>	✓	✓	✓	✓	X	X
<code>/=</code>	✓	X	X	X	X	X
<code>//=</code>	✓	X	X	X	X	X
<code>%=</code>	✓	X	X	X	X	X
<code>**=</code>	✓	X	X	X	X	X
<code>&=</code>	✓ (enteros)	X	X	X	✓	X
<code> =</code>	✓ (enteros)	X	X	X	✓	✓
<code>^=</code>	✓ (enteros)	X	X	X	✓	X

Ejemplo de asignación aumentada

`a = 8`

variables

objetos



Ejemplo de asignación aumentada

`a = 8`

`a+=6 #a = a +6`

variables

objetos



Ejemplo de asignación aumentada

`a = [1,2,3]`

variables

objetos

`a` → `[1,2,3]`

Ejemplo de asignación aumentada

```
a = [1,2,3]
```

```
a+=[7]    #a = [1,2,3] + [7]
```

variables

objetos

a → [1,2,3,7]

Ejemplo de asignación aumentada

`a = [1,2,3]`

`a = [1,2,3] + [7]`

variables

objetos

a

[1,2,3]

[1,2,3,7]



Funciones integradas de python (built-in functions)

- Python tiene una serie de funciones y tipos incorporados que están siempre disponibles
- <https://docs.python.org/3/library/functions.html>

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

Funciones integradas de python (built-in functions)

- Python tiene una serie de funciones y tipos incorporados que están siempre disponibles
- <https://docs.python.org/3/library/functions.html>

```
help(__builtins__)
```

```
print(dir(__builtins__))
```

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

Funciones integradas de python (built-in functions)

- Python tiene una serie de funciones y tipos incorporados que están siempre disponibles
- <https://docs.python.org/3/library/functions.html>

```
help(dir)
```

```
l = [1,2]
help(l)
```

```
print(dir(l))
```

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

Funciones integradas de python (built-in functions)

- Python tiene una serie de funciones y tipos incorporados que están siempre disponibles
- <https://docs.python.org/3/library/functions.html>

```
help(dir)
```

```
l = [1,2]
help(l)
```

```
print(dir(l))
```

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

Métodos

- Los **métodos** son funciones que están **asociadas a un objeto o clase** y que pueden operar sobre sus datos internos.

```
l = [1,2]  
help(l)
```

```
print(dir(l))
```

```
# key puede recibir cualquier función que reciba un  
# elemento de la lista y devuelva un valor que se pueda comparar.
```

```
l = [324,23,5432,2134354,1,21321,1232131232131312123231]  
l.sort(key=str)  
print(l)
```

El objeto range

- range genera una secuencia de números enteros:
`range(start, stop, step)`
- Objecto inmutable
- Soporta indexación, slicing, y longitud
- Eficiente: no almacena todos los números, los genera bajo demanda ([lazy evaluation](#)).
- No es un contenedor completo en el sentido clásico, porque no contiene todos sus elementos en memoria.

```
r = range(1_000_000_000)
print(type(r))
print(r)
print(r[-1])
```

El objeto range

- range genera una secuencia de números enteros:
range(start, stop, step)
- start, stop y step pueden ser negativos

```
1 print(list(range(-10,10)))  
2 print(list(range(10,-10,-1)))
```

La sentencia if

- Se utiliza para ejecutar un código solo si se cumple una condición

```
if condición:  
    # código que se ejecuta si condición es True
```

- if evalúa una expresión booleana.
- Si la condición es True se ejecuta el código
- Se pueden usar operadores lógicos, de comparación, pertenencia, identidad, etc
- Se pueden escribir en una sola línea.

```
if 5: print('a'); print(10); print(9)
```

La sentencia if

- Los paréntesis importan!

```
a = 5
b = -1
c = 18

if a > 0 or b > 0 and c < 0:
    print('hola')
```

Cláusulas `else` y `elif`

En la sentencia `if` la **expresión de control** nos devuelve un booleano, si es `True` se ejecutan las sentencias dentro del `if`, sin embargo, si deseamos ejecutar otro pedazo de código cuando la **expresión de control** es `False` necesitamos usar la cláusula `else`.

```
1 if <test>:  
2     <sentencia1>  
3 else:  
4     <sentencia2>  
5  
6 <sentencia3>
```

Si `<test>` es `True` se ejecuta `<sentencia1>`,
si `<test>` es `False` se ejecuta
`<sentencia2>`.

Independientemente de si se ejecuta
`<sentencia1>` o `<sentencia2>`, al salir del
`if` se va a ejecutar `<sentencia3>`.

Supongamos ahora que si <test> es **False** queremos ejecutar otro pedazo de código, pero queremos tener más de una alternativa, para esto utilizamos **elif**.

```
1 if <test1>:  
2     <sentencia1>  
3 elif <test2>:  
4     <sentencia2>  
5 elif <test3>  
6     <sentencia3>  
7     ...  
8 else:  
9     <sentencia4>  
10  
11 <sentencia5>
```

Si <test1> es **False** se ejecuta la siguiente **expresión de control** <test2>, si ésta es **True** se ejecuta <sentencia2> si es **False** saltamos a la **expresión de control** <test3>, y así sucesivamente.

Si todas las **expresiones de control** son **False** entonces al final se ejecuta el código dentro de **else**.

Podemos, escribir más de una sentencia en una sola línea, separadas por punto y coma. Si <test> es **True** se ejecutan todas las sentencias de la línea si <test> es **False** no se ejecuta ninguna.

```
1 if <test>: <sentencia1>; <sentencia2>; ...
```

Por ejemplo:

```
1 s = 'hola'
2
3 if s == 'hola': print (s[0]); print (s[1]); print (s[2]); print (s[3])
4 elif s == print (s[0]); print (s[1]); print (s[2])
5 else: print ('...'); print('...'); print('...')
```

elif y **else** también se puede escribir en una sola línea, pero no se recomienda a menos que sea una sentencia muy corta.

La expresión ternaria `if - else`

La expresión ternaria es una expresión que nos permite hacer lo mismo que una sentencia `if`, pero de una manera más sencilla. Su forma más general es:

```
1 A = Y if X else Z
```

Equivalente a :

```
1 if X:  
2     A = Y  
3 else:  
4     A = Z
```

Python ejecuta la expresión Y si y solo si X es `True` y ejecuta la expresión Z si y solo si X es `False` (short-circuit).

La sentencia **while** repite una acción una y otra vez mientras que una **expresión de control** se evalúe como **True**. La forma del ciclo while general es:

```
1 while <test>:  
2     <sentencia1>  
3  
4 <sentencia2>
```

<sentencia1> representa un bloque de código que se va a ejecutar una y otra vez mientras que <test> sea **True**. La **expresión de control** <test> puede involucrar una o más variables inicializadas antes del ciclo **while**, cuyos valores van a ir cambiando durante la ejecución del ciclo.

```
1 a = ['uno', 'dos', 'tres', 'cuatro']  
2 while a:  
3     a.pop( )  
4     print(a)
```

Sentencias break, continue y pass

Con el ciclo **while** se pueden usar las sentencias **break**, **continue** y **pass**. La forma general del ciclo es:

```
1 while <test1>:  
2     <sentencia1>:  
3     if <test2>: break  
4     if <test3>: continue
```

- ▶ **Break.** La sentencia `break` interrumpe/termina inmediatamente un ciclo. Después de que el ciclo es interrumpido la primer sentencia escrita después del ciclo se ejecuta.
- ▶ **Continue:** La sentencia `continue` inmediatamente termina la iteración en la que se encuentra el ciclo. Una vez que se interrumpe la iteración, se evalúa nuevamente la **expresión de control** y se determina si se va a seguir ejecutando el ciclo.
- ▶ **Pass:** La sentencia `pass` no hace nada. Es un placeholder vacío.

```
1 # ciclo infinito
2 while True:
3     pass
```

```
1 a = 0
2 while a :
3     a -= 1
4     if a%2: continue
5     print(f'{a} es par')
```

```
1 a = 10
2 while a:
3     a -= 1
4     if a == 3: break
5
6 print(f'el valor de a es {a}')
```

while - else

El ciclo `while` también se puede usar con la cláusula `else`:

```
1 while <test>:  
2     <sentencia1>  
3 else:  
4     <sentencia2>
```

En éste caso la sentencia dentro de `else` (<sentencia2>) se ejecuta cuando termina el ciclo `while`. Esto solo sucede si el ciclo termina una vez que la **expresión de control** se vuelve **False**, si el ciclo termina por una sentencia `break` entonces no se ejecuta <sentencia2>.

La sentencia for

- sirve para iterar sobre elementos de un objeto iterable, ejecutando un bloque de código una vez por cada elemento.

```
for variable in iterable:  
    # código
```

- Se puede correr con cualquier objeto iterable.

El ciclo **for** es un iterador de secuencias. Recorre los elementos de cualquier objeto que contenga una colección de objetos, por ejemplo cadenas, listas, tuplas, etc. La forma general del ciclo **for** es:

```
1 for <var> in <iterable>:  
2     <sentencia1>  
3     <sentencia2>  
4     ...
```

<iterable> es un objeto que contiene una colección de objetos, como: tuplas, listas, etc. La variable <var> va haciendo referencia a cada elemento de <iterable>, uno por uno. Para cada elemento (<var>) se ejecutan las sentencias dentro del ciclo **for**.

El ciclo **for** también se puede usar con la cláusula **else**. El principio de funcionamiento es el mismo que con la cláusula **while** el las sentencias dentro de **else** se ejecutan si el ciclo **for** termina. Sin embargo **else** no se ejecuta si el **for** termina con un **break**:

```
1 for <variable> in <iterable>:  
2     <sentencia1>  
3 else:  
4     <sentencia2>
```

El ciclo **for** puede tener otros **for** anidados, o **while**, o **if**.

La sentencia for

Ejemplos:

- Podemos iterar una cadena
- Podemos iterar una lista (es igual con la tupla)
- Podemos iterar en x,y para una colección de tuplas.
- Podemos iterar diccionarios, aquí hay varias maneras:
 - Iterar las claves y acceder a cada valor por `d[clave]`
 - Iterar `d.values()`
- Los range también son iterables
- Podemos iterar un enumerate
- Podemos usar `zip()` para iterar sobre varios iterables al mismo tiempo.
- Se puede iterar usando `_`

El operador walrus

- Es un operador que asigna y devuelve el valor en la misma expresión.

```
# Walrus operator (asignar y usar en la misma línea)
if (n := len("hola")) > 3:
    print(f"tiene {n} letras")
```

unpacking

- desempaquetar una estructura en múltiples variables.
- La de variables debe coincidir
- Podemos:
 - Asignar elementos de una secuencia a variables individuales
 - Desempaquetar con * para capturar múltiples elementos
 - Desempaquetar ignorando valores con _
 - Hacer un desempaquetado anidado
 - En ciclos y funciones también se hace desempaquetado.

Ejemplos de unpacking

```
# Capturar el resto en una lista
primero, *resto = [1, 2, 3, 4, 5]
# primero=1, resto=[2, 3, 4, 5]

# Al inicio
*inicio, ultimo = [1, 2, 3, 4, 5]
# inicio=[1, 2, 3, 4], ultimo=5

# En medio
primero, *medio, ultimo = [1, 2, 3, 4, 5]
# primero=1, medio=[2, 3, 4], ultimo=5

# El * siempre produce una lista, aunque esté vacía
a, *b, c = [1, 2]
# a=1, b=[], c=2
```

```
# Ignorar un valor
a, _, c = (1, 2, 3) # a=1, c=3, el 2 se descarta

# Ignorar varios
a, *_ , z = [1, 2, 3, 4, 5] # a=1, z=5, lo del medio se descarta

# Múltiples ignorados
_, segundo, _ = (1, 2, 3) # segundo=2
```

```
# Tuplas dentro de tuplas
(a, b), c = ((1, 2), 3)
# a=1, b=2, c=3

# Listas dentro de listas
[x, [y, z]] = [1, [2, 3]]
# x=1, y=2, z=3

# Más profundo
a, (b, (c, d)) = (1, (2, (3, 4)))
# a=1, b=2, c=3, d=4
```

Comprensión de listas

- Una comprensión es básicamente una forma compacta de escribir un bucle que construye una colección.
- Una comprensión de listas son corchetes cuadrados rodeando una expresión seguida de la declaración for y luego cero o más declaraciones for o if.

[<expresión> for <variable> in <iterable> if <condición>]

[<expresión> for <variable> in <iterable> for <variable2> in <iterable2> if <condición>]

Ejercicios:

- Calcula el cuadrado de los números del 1 al 10.
- Filtra los números pares del 1 al 10.
- Transforma una lista de strings a mayúsculas

Comprensión de otros objetos

- También se puede hacer comprensión de diccionarios y sets.

Ejercicios:

- Genera una lista de tuplas (x,y) donde x son números del 0 al 4 mayores que 2, e y son números del 0 al 4 menores que 3.

Salida:

`[(3, 0), (3, 1), (3, 2), (4, 0), (4, 1), (4, 2)]`

- Dada la lista ["sol", "luna", "estrella"], crea un diccionario donde cada palabra sea la clave y su longitud sea el valor.
- Dado el diccionario {"a": 1, "b": 2}, invierte sus claves y valores.
- Dada la lista [1, -1, 2, -2, 1], genera un set con los valores absolutos de cada número usando comprensión de sets.
- Dadas las listas ["rojo", "azul"] y ["S", "M"], genera todas las combinaciones posibles de color y talla como tuplas, usando comprensión de listas

Ejercicios/ejemplos

Ejemplos:

- Busca/inventa la temperatura (°C) y velocidad del viento (km/h) de al menos 4 ciudades. Calcula la sensación térmica de cada una usando la fórmula simplificada:

$$\text{sensacion} = 13.12 + 0.6215 * T - 11.37 * (V^{**0.16}) + 0.3965 * T * (V^{**0.16})$$

donde T = temperatura, V = velocidad del viento

- Dada una lista de temperaturas, cuenta cuántos días superaron los 25°C.

`temps_julio = [23, 26, 28, 24, 27, 30, 25, 22, 29, 31]`

- Dada esta lista de precipitación diaria (mm):

`lluvia = [0, 0, 3.2, 15.5, 8.1, 0, 0, 1.2, 0, 55.3]`

Calcula: total acumulado, días con lluvia, día con máxima precipitación, y clasifica cada día según intensidad.

- ▶ Iteración: proceso de recorrer los elementos de una colección u objetos iterables.
- ▶ Iterable: Un objeto iterable es aquel que puede regresar sus miembros uno a la vez.

Las secuencias son iterables, por ejemplo, las listas, tuplas y cadenas. Los diccionarios y los archivos también son iterables. Los objetos iterables se pueden usar en los ciclo **for** y en cualquier otro lugar que necesite una secuencia.

- Iterador: Un iterador es que es un objeto que representa un flujo de datos. Se puede entender también como un objeto que es usado para iterar un objeto iterable. El objeto se inicializa usando el método `iter()`. La función integrada `next()` se usa para obtener el siguiente valor del iterador, si se han devuelto todos los elementos del iterable, `next()` devuelve una excepción: `StopIteration`. Con los iteradores no podemos ir hacia atrás.

Más info en <https://docs.python.org/dev/glossary.html>

```
1 colores = ['rojo', 'rosa', 'azul', 'amarillo', 'verde']
2 iterador = iter(colores)
3 type(iterador)
4
5 next(iterador) # regresa rojo
6 next(iterador) #regresa rosa
```

Podemos crear dos iteradores con el mismo objeto iterable.

```
1  electrodomésticos = ('microondas', 'licuadora', 'tostador')
2
3  # creamos dos iterados
4  iterador_1 = iter(electrodomésticos)
5      print(next(iterador_1))
6
7  iterador_2 = iter(electrodomésticos)
8      print(next(iterador_1))
9      print(next(iterador_2))
```

Los iterables regresan un solo valor del iterable cuando se le pide con la función **next()**, sin embargo podemos pedir todos los valores en un solo paso si usamos las funciones integradas **list()**, **tuple()** y **set()**.

```
1 electrodomésticos = ('microondas', 'licuadora', 'tostador')
2 iterador = iter(electrodomesticos)
3 lista_electrodomesticos = list(iterador)
4     print(lista_electrodomesticos)
```

El ciclo **for** llama a **iter()** para obtener un iterador del iterable. Después llama la función **next()** repetidamente para obtener cada elemento del iterador. Termina cuando **next()** regresa la excepción **StopIteration**.